Mini project report on

Digital Antique Provenance and Auction Platform

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE23CS351A – DBMS Project

Submitted by:

Shivanand	**PES2UG24CS472**
Shreenandan Jamadar	**PES2UG24CS479**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Digital Antique Provenance and Auction Platform

is a bonafide work carried out by

Shivanand	**PES2UG24CS472**
Shreenandan Jamadar	**PES2UG24CS479**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CSS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Digital Antique Provenance and Auction Platform** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Shivanand	**PES2UG24CS472**	**<Signature>**
Shreenandan Jamadar	**PES2UG24CS479**	**<Signature>**

ACKNOWLEDGEMENT

We would like to express my gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A - DBMS Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from my family and friends.

ABSTRACT

Antique collections and auctions are traditionally recorded in notebooks, spreadsheets and unconnected systems. Because of this, the details of an antique, the chain of owners it has passed through, its valuation, the auction it is listed in, the bids placed on it and the payment made for it are hard to keep consistent and hard to verify.

This project presents the **Digital Antique Provenance and Auction Platform**, a relational database that stores all of this information in one organised and reliable place. The system is modelled with an Entity-Relationship diagram of nine entities (Owner, Antique Item, Provenance, Valuation, Auction, Bidder, Bid, Transaction and Payment) and mapped to a relational schema in which primary and foreign keys enforce data integrity.

The database, antique_auction_db, was implemented in MySQL. The report covers the DDL used to create the nine tables, DML statements that load sample data, queries using joins, GROUP BY, HAVING, aggregate functions, correlated and nested sub-queries, UPDATE and DELETE operations, a view, transaction control (COMMIT), a stored procedure, a user-defined function and a trigger that automatically marks an antique as "Auctioned" when it is listed in an auction.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	11
2.	PROBLEM DEFINITION	13
3.	ER MODEL	14
4.	ER TO RELATIONAL MAPPING	16
5.	DDL STATEMENTS	18
6.	DML STATEMENTS	25
7.	QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)	32
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	39
9.	FRONT END DEVELOPMENT	44
GITHUB LINK		49
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		50

LIST OF TABLES

Table No.	Title	Page No.
3.1	Entities and their attributes	**14**
4.1	Tables of antique_auction_db with their primary and foreign keys	**17**

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	ER diagram of the Digital Antique Provenance and Auction Platform	**14**
4.1	Relational schema diagram	**17**
5.1	Creating the database antique_auction_db	**18**
5.2	Selecting the database	**18**
5.3	OWNER table (owner_id is the primary key)	**18**
5.4	ANTIQUE_ITEM table (owner_id references OWNER)	**19**
5.5	PROVENANCE table (item_id references ANTIQUE_ITEM)	**19**
5.6	VALUATION table (item_id references ANTIQUE_ITEM)	**20**
5.7	AUCTION table (item_id references ANTIQUE_ITEM)	**20**
5.8	BIDDER table	**21**
5.9	BID table (auction_id and bidder_id are foreign keys)	**21**
5.10	TRANSACTION table (final auction result)	**22**
5.11	PAYMENT table (payment against each transaction)	**22**
5.12	Complete table structure – all 9 tables in antique_auction_db	**23**
5.13	ALTER TABLE – add column auction_type to AUCTION	**23**
5.14	ALTER TABLE – modify the description column of AUCTION	**24**
5.15	CREATE VIEW auction_details	**24**
6.1	INSERT statements and records of OWNER	**25**
6.2	INSERT statements and records of ANTIQUE_ITEM	**26**
6.3	INSERT statements and records of PROVENANCE	**26**
6.4	INSERT statements and records of VALUATION	**27**
6.5	INSERT statements and records of AUCTION	**28**
6.6	INSERT statements and records of BIDDER	**29**
6.7	INSERT statements and records of BID	**30**
6.8	INSERT statements and records of TRANSACTION	**30**
6.9	INSERT statements and records of PAYMENT	**31**
7.1	JOIN query – all bids with auction, item and bidder	**32**
7.2	GROUP BY with COUNT – bids per auction	**33**
7.3	GROUP BY, SUM and HAVING	**34**

7.4	Aggregate functions on bid amounts	**34**
7.5	MAX with GROUP BY – highest bid per auction	**35**
7.6	UPDATE inside START TRANSACTION and COMMIT	**36**
7.7	DELETE operation	**37**
7.8	Nested query – items above the average estimated value	**38**
8.1	Creating and calling the procedure GetAuctionDetails()	**39**
8.2	SHOW PROCEDURE STATUS	**39**
8.3	Creating the function CalculateFutureValue()	**40**
8.4	Using the function on the VALUATION table	**40**
8.5	SHOW FUNCTION STATUS	**41**
8.6	Creating the trigger after_auction_insert	**41**
8.7	Testing the trigger – a test auction is inserted for item 7	**42**
8.8	Trigger result – current_status of item 7 is now 'Auctioned'	**42**
8.9	SHOW TRIGGERS – trigger on the AUCTION table	**43**
9.1	Sign in page – an existing user signs in with email and password.	**44**
9.2	Create account page – a new user registers with full name, email and password.	**44**
9.3	Home page – introduction with the total antiques, active auctions and total bids.	**45**
9.4	Featured antiques on the home page, with search by name and a category filter.	**45**
9.5	Upcoming auctions (with status filter) and latest feedback on the home page.	**45**
9.6	Antiques page – all antiques with year, origin, condition and estimated value.	**46**
9.7	Auctions page – upcoming and live auctions with a status filter.	**46**
9.8	User feedback page – average rating, a form to submit feedback, and community reviews.	**46**
9.9	Auction details page – item details, previous highest bid, minimum next bid and the Place a bid form.	**47**
9.10	My bids page – the bids placed by the signed-in user.	**47**
9.11	Admin dashboard – recorded activities, total bids and total auctions, and inventory by category.	**47**

9.12 Admin dashboard – auctions by status and activity by action. **48**
9.13 Admin dashboard – most viewed auctions and most viewed antiques. **48**
9.14 Admin dashboard – recent activity log (login, logout and page views). **48**

1. INTRODUCTION

Antiques are valuable not only because of their age or material, but because of their history. Who made an item, where it came from, who owned it before and what experts think it is worth are all facts that decide its price at an auction. In many places this information is still maintained manually or in separate registers, which makes it difficult to search, easy to lose and impossible to verify quickly.

The **Digital Antique Provenance and Auction Platform** is a database-centred solution to this problem. It keeps the details of antiques, their owners, ownership history (provenance), valuations, auctions, bidders, bids, transactions and payments in a single relational database, with the relationships between them enforced by primary and foreign keys.

1.1 Objectives

- To maintain complete details of antique items.
- To store owner information and ownership details.
- To maintain the provenance and previous ownership history of antiques.
- To record antique valuation and expert appraisal details.
- To manage auction information, including dates, location and base price.
- To maintain bidder information and their registration details.
- To record and manage bids placed during auctions.
- To identify the winning bidder and the final transaction price.
- To maintain payment records for completed transactions.
- To use primary keys and foreign keys to maintain data integrity and relationships between tables.

1.2 Tools and Technologies

- **Database:** MySQL (database name: antique_auction_db).
- **Language:** SQL – DDL, DML, queries, views, stored procedures, functions and triggers.
- **Interface:** MySQL command-line client.
- **Version control:** GitHub.

1.3 Team

Group No. 29, Section H (5th semester): Shivanand (PES2UG24CS472) and Shreenandan Jamadar (PES2UG24CS479).

2. PROBLEM DEFINITION

2.1 Problem Statement

Traditional antique collection and auction records are often maintained manually or in separate systems. This makes it difficult to manage antique details, ownership history, provenance, valuation, auctions, bidders, bids, transactions, and payments efficiently.

The proposed Digital Antique Provenance and Auction Platform provides a centralized database to store and manage all this information in an organized and reliable way.

2.2 Requirements of the System

- Store each antique with its category, description, year made, origin and condition, and link it to its owner.
- Keep a history of provenance records (previous owner, period and method of acquisition) for every antique.
- Record one or more expert valuations for each antique.
- Create auctions for antiques with a start date, end date, location, base price and status.
- Register bidders and record every bid with its amount and time.
- Record the final transaction (winning bidder and final price) of a completed auction.
- Record the payment made against each transaction.

2.3 Scope

The project covers the design and implementation of the database: ER modelling, relational mapping, table creation, data insertion, querying, and database programming (view, transaction, procedure, function and trigger). It is operated through the MySQL command-line client.

3. ER MODEL

The Entity-Relationship (ER) diagram represents the structure of the Digital Antique Provenance and Auction Platform. Each rectangle is an entity, each oval an attribute and each diamond a relationship, with the cardinality (1 or N) marked on the connecting lines.

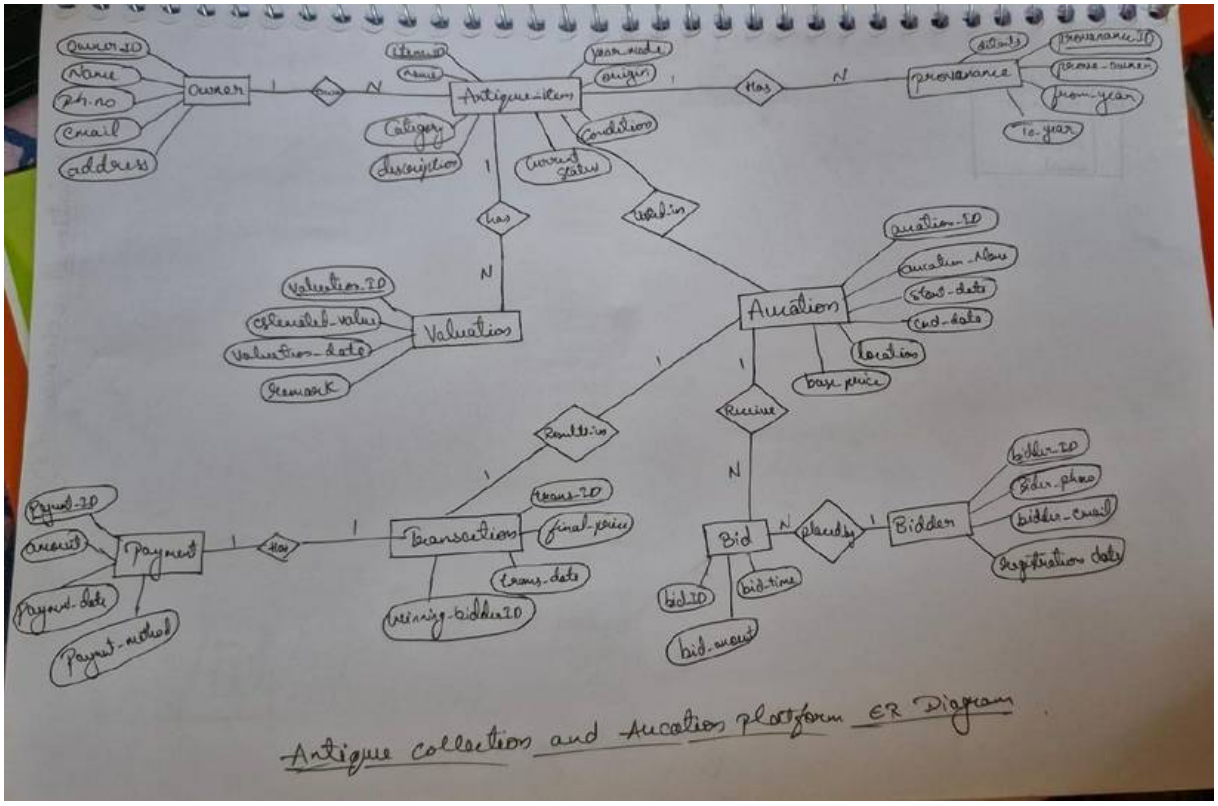


Figure 3.1: ER diagram of the Digital Antique Provenance and Auction Platform

3.1 Entities and Attributes

Table 3.1: Entities and their attributes

Entity	Description	Attributes
OWNER	Stores antique owner's details	owner_id, name, phone, email, address
ANTIQUE_ITEM	Stores information about each antique	item_id, name, category, description, year_made, origin, condition_status, current_status
PROVENANCE	Historical ownership and origin of an antique	provenance_id, previous_owner, from_year, to_year, acquisition_method, details
VALUATION	Expert valuation details	valuation_id, estimated_value, valuation_date, expert_id, expert_name, method, remarks

AUCTION	Auction information	auction_id, name, description, start_date, end_date, location, base_price, status
BIDDER	Bidder details	bidder_id, name, phone, email, address, registration_date
BID	Bids placed by bidders	bid_id, bid_amount, bid_time
TRANSACTION	Final auction transaction	transaction_id, final_price, transaction_date
PAYMENT	Payment information	payment_id, amount, payment_date, payment_method, status

3.2 Relationships

- **OWNER – ANTIQUE_ITEM (owns, 1:N):** one owner can own many antique items.
- **ANTIQUE_ITEM – PROVENANCE (has, 1:N):** one antique can have many provenance records.
- **ANTIQUE_ITEM – VALUATION (has, 1:N):** one antique can have many valuation records.
- **ANTIQUE_ITEM – AUCTION (listed in):** an antique item can be listed in an auction.
- **AUCTION – BID (receives, 1:N):** one auction can receive many bids.
- **BIDDER – BID (placed by, 1:N):** one bidder can place many bids.
- **AUCTION – TRANSACTION (results in, 1:1):** a completed auction results in one transaction.
- **TRANSACTION – PAYMENT (has, 1:1):** one transaction has one payment record.

4. ER TO RELATIONAL MAPPING

4.1 STEPS OF ALGORITHM FOR CHOOSEN PROBLEM

The ER diagram was converted into relational tables using the standard ER-to-relational mapping algorithm. The steps and their outcome for this project are:

Step 1 – Mapping of regular entity types. Each strong entity becomes a table whose primary key is the key attribute of the entity: OWNER(owner_id), ANTIQUE_ITEM(item_id), PROVENANCE(provenance_id), VALUATION(valuation_id), AUCTION(auction_id), BIDDER(bidder_id), BID(bid_id), TRANSACTION(transaction_id) and PAYMENT(payment_id). All simple attributes are included as columns.

Step 2 – Mapping of weak entity types. The design has no weak entity types, because every entity has its own key attribute.

Step 3 – Mapping of binary 1:1 relationships. AUCTION–TRANSACTION and TRANSACTION–PAYMENT are 1:1. The foreign-key approach is used: auction_id is placed in TRANSACTION and transaction_id is placed in PAYMENT.

Step 4 – Mapping of binary 1:N relationships. The primary key of the "1" side is added as a foreign key on the "N" side: owner_id in ANTIQUE_ITEM; item_id in PROVENANCE, VALUATION and AUCTION; auction_id and bidder_id in BID.

Step 5 – Mapping of binary M:N relationships. Bidders and auctions are related many-to-many (a bidder bids in many auctions and an auction has many bidders). This is resolved by the BID entity, which carries both auction_id and bidder_id as foreign keys.

Step 6 – Mapping of multivalued attributes. There are no multivalued attributes, so no extra tables are needed.

Step 7 – Mapping of n-ary relationships. There are no relationships of degree higher than two. The winning bidder of a transaction is stored as the foreign key winning_bidder_id referencing BIDDER.

4.2 COMPLETE DIAGRAM OF RELATIONAL MAPPING

The relational schema converts the ER diagram into tables. Each entity becomes a table, and relationships are represented using primary keys (PK) and foreign keys (FK).

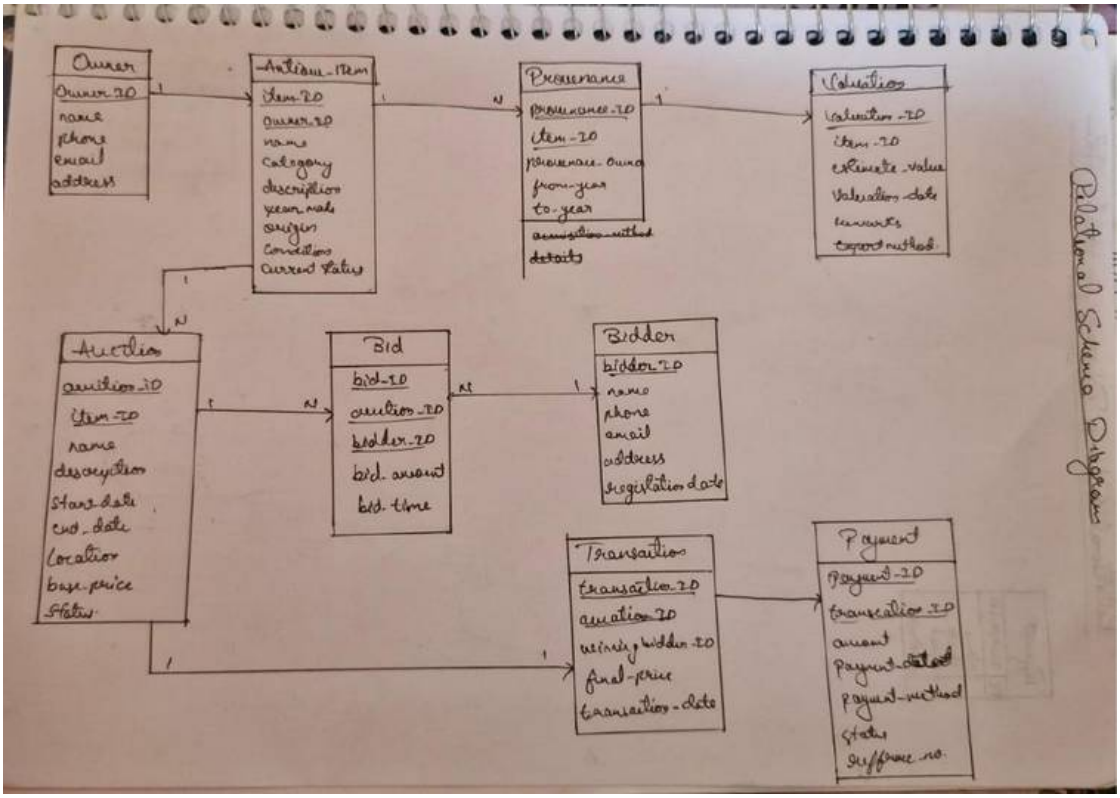


Figure 4.1: Relational schema diagram

Table 4.1: Tables of antique_auction_db with their primary and foreign keys

Table	Primary Key	Foreign Key(s)
OWNER	owner_id	—
ANTIQUE_ITEM	item_id	owner_id → OWNER
PROVENANCE	provenance_id	item_id → ANTIQUE_ITEM
VALUATION	valuation_id	item_id → ANTIQUE_ITEM
AUCTION	auction_id	item_id → ANTIQUE_ITEM
BIDDER	bidder_id	—
BID	bid_id	auction_id → AUCTION; bidder_id → BIDDER
TRANSACTION	transaction_id	auction_id → AUCTION; winning_bidder_id → BIDDER
PAYMENT	payment_id	transaction_id → TRANSACTION

5. DDL STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE CREATION

The database **antique_auction_db** was created and selected, followed by the nine tables. Each CREATE TABLE statement and the DESC output that confirms its structure is shown below.

5.1 Creating the database and tables

```
PES2UG24CS472_PES2UG24CS479 > CREATE DATABASE antique_auction_db;
Query OK, 1 row affected (0.0224 sec)
PES2UG24CS472_PES2UG24CS479 > USE antique_auction_db;
Default schema set to `antique_auction_db`.
Fetching global names, object names from `antique_auction_db
```

Figure 5.1: Creating the database antique_auction_db

```
PES2UG24CS472_PES2UG24CS479 > show databases;
+--------------------+
| Database           |
+--------------------+
| antique_auction_db |
| college            |
| company            |
```

Figure 5.2: Selecting the database

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE OWNER (
                          ->     owner_id INT PRIMARY KEY,
                          ->     name VARCHAR(100) NOT NULL,
                          ->     phone VARCHAR(15),
                          ->     email VARCHAR(100),
                          ->     address VARCHAR(255)
                          -> );
Query OK, 0 rows affected (0.0439 sec)
PES2UG24CS472_PES2UG24CS479 > desc owner;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| owner_id | int          | NO   | PRI | NULL    |       |
| name     | varchar(100) | NO   |     | NULL    |       |
| phone    | varchar(15)  | YES  |     | NULL    |       |
| email    | varchar(100) | YES  |     | NULL    |       |
| address  | varchar(255) | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
5 rows in set (0.0207 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.3: OWNER table (owner_id is the primary key)

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE ANTIQUE_ITEM (
                          ->     item_id INT PRIMARY KEY,
                          ->     owner_id INT NOT NULL,
                          ->     name VARCHAR(100) NOT NULL,
                          ->     category VARCHAR(50),
                          ->     description TEXT,
                          ->     year_made INT,
                          ->     origin VARCHAR(100),
                          ->     condition_status VARCHAR(100),
                          ->     current_status VARCHAR(50),
                          ->
                          ->     FOREIGN KEY (owner_id) REFERENCES OWNER(owner_id
                          -> );
Query OK, 0 rows affected (0.0675 sec)
PES2UG24CS472_PES2UG24CS479 > desc antique_item;
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| item_id          | int          | NO   | PRI | NULL    |       |
| owner_id         | int          | NO   | MUL | NULL    |       |
| name             | varchar(100) | NO   |     | NULL    |       |
| category         | varchar(50)  | YES  |     | NULL    |       |
| description      | text         | YES  |     | NULL    |       |
| year_made        | int          | YES  |     | NULL    |       |
| origin           | varchar(100) | YES  |     | NULL    |       |
| condition_status | varchar(100) | YES  |     | NULL    |       |
| current_status   | varchar(50)  | YES  |     | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
9 rows in set (0.0029 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.4: ANTIQUE_ITEM table (owner_id references OWNER)

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE PROVENANCE (
                          ->     provenance_id INT PRIMARY KEY,
                          ->     item_id INT NOT NULL,
                          ->     previous_owner VARCHAR(100),
                          ->     from_year INT,
                          ->     to_year INT,
                          ->     acquisition_method VARCHAR(100),
                          ->     details TEXT,
                          ->
                          ->     FOREIGN KEY (item_id) REFERENCES ANTIQUE_ITEM(it
                          -> );
Query OK, 0 rows affected (0.0540 sec)
PES2UG24CS472_PES2UG24CS479 > desc provenance;
+--------------------+--------------+------+-----+---------+-------+
| Field              | Type         | Null | Key | Default | Extra |
+--------------------+--------------+------+-----+---------+-------+
| provenance_id      | int          | NO   | PRI | NULL    |       |
| item_id            | int          | NO   | MUL | NULL    |       |
| previous_owner     | varchar(100) | YES  |     | NULL    |       |
| from_year          | int          | YES  |     | NULL    |       |
| to_year            | int          | YES  |     | NULL    |       |
| acquisition_method | varchar(100) | YES  |     | NULL    |       |
| details            | text         | YES  |     | NULL    |       |
+--------------------+--------------+------+-----+---------+-------+
7 rows in set (0.0028 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.5: PROVENANCE table (item_id references ANTIQUE_ITEM)

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE VALUATION (
                          ->     valuation_id INT PRIMARY KEY,
                          ->     item_id INT NOT NULL,
                          ->     estimated_value DECIMAL(12,2),
                          ->     valuation_date DATE,
                          ->     expert_id INT,
                          ->     expert_name VARCHAR(100),
                          ->     method VARCHAR(50),
                          ->     remarks TEXT,
                          ->
                          ->     FOREIGN KEY (item_id) REFERENCES ANTIQUE_ITEM(it
                          -> );
Query OK, 0 rows affected (0.0827 sec)
PES2UG24CS472_PES2UG24CS479 > desc valuation;
+-----------------+---------------+------+-----+---------+-------+
| Field           | Type          | Null | Key | Default | Extra |
+-----------------+---------------+------+-----+---------+-------+
| valuation_id    | int           | NO   | PRI | NULL    |       |
| item_id         | int           | NO   | MUL | NULL    |       |
| estimated_value | decimal(12,2) | YES  |     | NULL    |       |
| valuation_date  | date          | YES  |     | NULL    |       |
| expert_id       | int           | YES  |     | NULL    |       |
| expert_name     | varchar(100)  | YES  |     | NULL    |       |
| method          | varchar(50)   | YES  |     | NULL    |       |
| remarks         | text          | YES  |     | NULL    |       |
+-----------------+---------------+------+-----+---------+-------+
8 rows in set (0.0025 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.6: VALUATION table (item_id references ANTIQUE_ITEM)

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE AUCTION (
                          ->     auction_id INT PRIMARY KEY,
                          ->     item_id INT NOT NULL,
                          ->     name VARCHAR(100) NOT NULL,
                          ->     description TEXT,
                          ->     start_date DATE,
                          ->     end_date DATE,
                          ->     location VARCHAR(100),
                          ->     base_price DECIMAL(12,2),
                          ->     status VARCHAR(50),
                          ->
                          ->     FOREIGN KEY (item_id) REFERENCES ANTIQUE_ITEM(it
                          -> );
Query OK, 0 rows affected (0.0733 sec)
PES2UG24CS472_PES2UG24CS479 > desc auction;
+-------------+---------------+------+-----+---------+-------+
| Field       | Type          | Null | Key | Default | Extra |
+-------------+---------------+------+-----+---------+-------+
| auction_id  | int           | NO   | PRI | NULL    |       |
| item_id     | int           | NO   | MUL | NULL    |       |
| name        | varchar(100)  | NO   |     | NULL    |       |
| description | text          | YES  |     | NULL    |       |
| start_date  | date          | YES  |     | NULL    |       |
| end_date    | date          | YES  |     | NULL    |       |
| location    | varchar(100)  | YES  |     | NULL    |       |
| base_price  | decimal(12,2) | YES  |     | NULL    |       |
| status      | varchar(50)   | YES  |     | NULL    |       |
+-------------+---------------+------+-----+---------+-------+
9 rows in set (0.0025 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.7: AUCTION table (item_id references ANTIQUE_ITEM)

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE BIDDER (
                          ->     bidder_id INT PRIMARY KEY,
                          ->     name VARCHAR(100) NOT NULL,
                          ->     phone VARCHAR(15),
                          ->     email VARCHAR(100),
                          ->     address VARCHAR(255),
                          ->     registration_date DATE
                          -> );
Query OK, 0 rows affected (0.0458 sec)
PES2UG24CS472_PES2UG24CS479 > desc bidder;
+-------------------+--------------+------+-----+---------+-------+
| Field             | Type         | Null | Key | Default | Extra |
+-------------------+--------------+------+-----+---------+-------+
| bidder_id         | int          | NO   | PRI | NULL    |       |
| name              | varchar(100) | NO   |     | NULL    |       |
| phone             | varchar(15)  | YES  |     | NULL    |       |
| email             | varchar(100) | YES  |     | NULL    |       |
| address           | varchar(255) | YES  |     | NULL    |       |
| registration_date | date         | YES  |     | NULL    |       |
+-------------------+--------------+------+-----+---------+-------+
6 rows in set (0.0024 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.8: BIDDER table

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE BID (
                          ->     bid_id INT PRIMARY KEY,
                          ->     auction_id INT NOT NULL,
                          ->     bidder_id INT NOT NULL,
                          ->     bid_amount DECIMAL(12,2) NOT NULL,
                          ->     bid_time DATETIME,
                          ->
                          ->     FOREIGN KEY (auction_id) REFERENCES AUCTION(auct
                          ->     FOREIGN KEY (bidder_id) REFERENCES BIDDER(bidder
                          -> );
Query OK, 0 rows affected (0.0424 sec)
PES2UG24CS472_PES2UG24CS479 > desc did;
ERROR: 1146: Table 'antique_auction_db.did' doesn't exist
PES2UG24CS472_PES2UG24CS479 > desc bid;
+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| bid_id     | int           | NO   | PRI | NULL    |       |
| auction_id | int           | NO   | MUL | NULL    |       |
| bidder_id  | int           | NO   | MUL | NULL    |       |
| bid_amount | decimal(12,2) | NO   |     | NULL    |       |
| bid_time   | datetime      | YES  |     | NULL    |       |
+------------+---------------+------+-----+---------+-------+
5 rows in set (0.0022 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.9: BID table (auction_id and bidder_id are foreign keys)

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE TRANSACTION (
                          ->     transaction_id INT PRIMARY KEY,
                          ->     auction_id INT NOT NULL,
                          ->     winning_bidder_id INT NOT NULL,
                          ->     final_price DECIMAL(12,2) NOT NULL,
                          ->     transaction_date DATE,
                          ->
                          ->     FOREIGN KEY (auction_id) REFERENCES AUCTION(auct
                          ->     FOREIGN KEY (winning_bidder_id) REFERENCES BIDDE
                          -> );
Query OK, 0 rows affected (0.0817 sec)
PES2UG24CS472_PES2UG24CS479 > desc transaction;
+-------------------+---------------+------+-----+---------+-------+
| Field             | Type          | Null | Key | Default | Extra |
+-------------------+---------------+------+-----+---------+-------+
| transaction_id    | int           | NO   | PRI | NULL    |       |
| auction_id        | int           | NO   | MUL | NULL    |       |
| winning_bidder_id | int           | NO   | MUL | NULL    |       |
| final_price       | decimal(12,2) | NO   |     | NULL    |       |
| transaction_date  | date          | YES  |     | NULL    |       |
+-------------------+---------------+------+-----+---------+-------+
5 rows in set (0.0024 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.10: TRANSACTION table (final auction result)

```
PES2UG24CS472_PES2UG24CS479 > CREATE TABLE PAYMENT (
                          ->     payment_id INT PRIMARY KEY,
                          ->     transaction_id INT NOT NULL,
                          ->     amount DECIMAL(12,2) NOT NULL,
                          ->     payment_date DATE,
                          ->     payment_method VARCHAR(50),
                          ->     status VARCHAR(50),
                          ->
                          ->     FOREIGN KEY (transaction_id) REFERENCES TRANSACT
                          -> );
Query OK, 0 rows affected (0.0835 sec)
PES2UG24CS472_PES2UG24CS479 > desc payment;
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| payment_id     | int           | NO   | PRI | NULL    |       |
| transaction_id | int           | NO   | MUL | NULL    |       |
| amount         | decimal(12,2) | NO   |     | NULL    |       |
| payment_date   | date          | YES  |     | NULL    |       |
| payment_method | varchar(50)   | YES  |     | NULL    |       |
| status         | varchar(50)   | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
6 rows in set (0.0023 sec)
PES2UG24CS472_PES2UG24CS479 >
```

Figure 5.11: PAYMENT table (payment against each transaction)

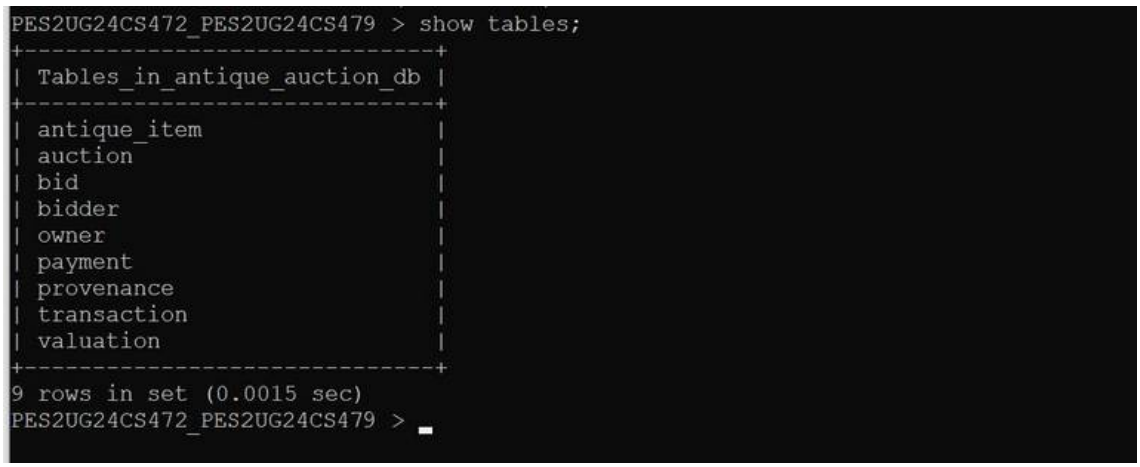


Figure 5.12: Complete table structure – all 9 tables in antique_auction_db

5.2 Additional DDL commands

ALTER TABLE was used to add the column auction_type (default 'Physical') to AUCTION, and to change the description column of AUCTION to VARCHAR(500). A view, auction_details, combines auction and item information.

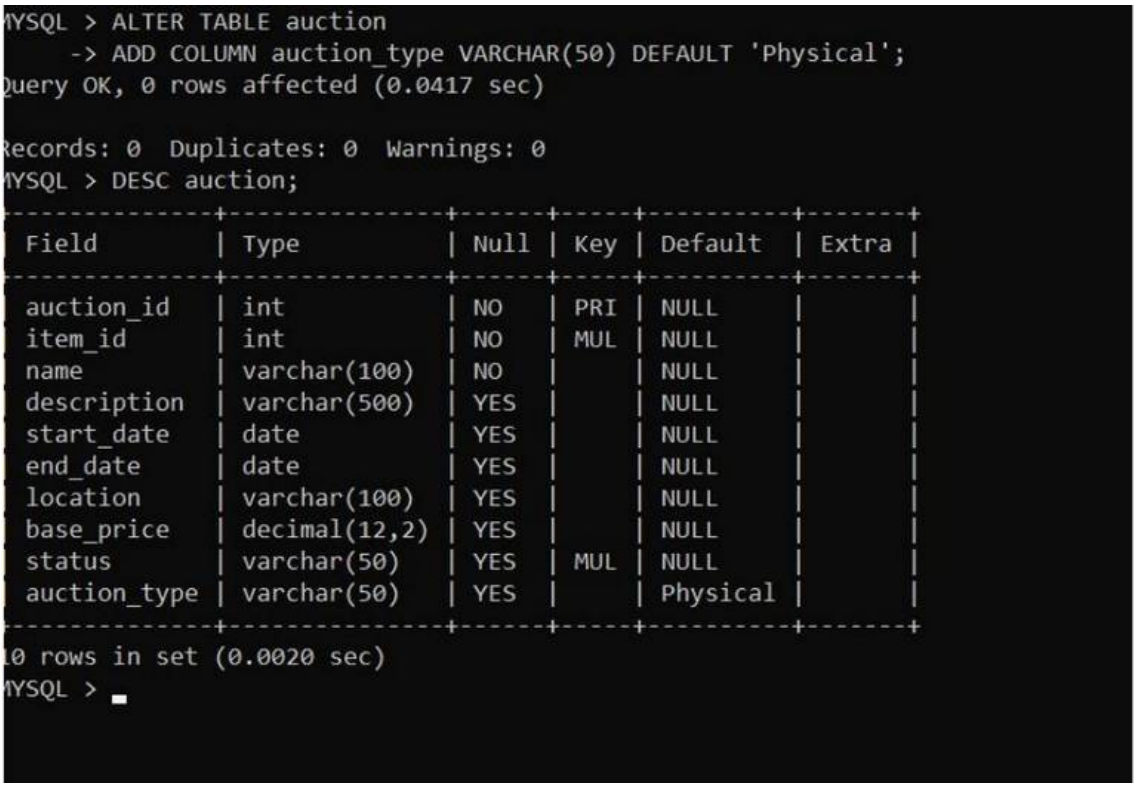


Figure 5.13: ALTER TABLE – add column auction_type to AUCTION

```
MYSQL > ALTER TABLE auction
     -> MODIFY description VARCHAR(500);
Query OK, 0 rows affected (0.0504 sec)

Records: 0  Duplicates: 0  Warnings: 0
MYSQL > DESC auction;
+--------------+---------------+------+-----+-----------+-------+
| Field        | Type          | Null | Key | Default   | Extra |
+--------------+---------------+------+-----+-----------+-------+
| auction_id   | int           | NO   | PRI | NULL      |       |
| item_id      | int           | NO   | MUL | NULL      |       |
| name         | varchar(100)  | NO   |     | NULL      |       |
| description  | varchar(500)  | YES  |     | NULL      |       |
| start_date   | date          | YES  |     | NULL      |       |
| end_date     | date          | YES  |     | NULL      |       |
| location     | varchar(100)  | YES  |     | NULL      |       |
| base_price   | decimal(12,2) | YES  |     | NULL      |       |
| status       | varchar(50)   | YES  | MUL | NULL      |       |
| auction_type | varchar(50)   | YES  |     | Physical  |       |
+--------------+---------------+------+-----+-----------+-------+
10 rows in set (0.0018 sec)
MYSQL >
```

Figure 5.14: ALTER TABLE – modify the description column of AUCTION

```
MYSQL > CREATE VIEW auction_details AS
    -> SELECT
    ->     a.auction_id,
    ->     a.name AS auction_name,
    ->     ai.name AS item_name,
    ->     a.base_price,
    ->     a.start_date,
    ->     a.end_date,
    ->     a.status
    -> FROM auction a
    -> JOIN antique_item ai
    -> ON a.item_id = ai.item_id;
Query OK, 0 rows affected (0.0227 sec)
MYSQL > SELECT * FROM auction_details;
+------------+-------------------------------+-------------------------+-----------+------------+------------+----------+
| auction_id | auction_name                  | item_name               | base_price | start_date | end_date   | status   |
+------------+-------------------------------+-------------------------+-----------+------------+------------+----------+
|          1 | Mysore Heritage Auction       | Mysore Palace Painting  |   20000.00 | 2026-10-10 | 2026-10-10 | Upcoming |
|          2 | Traditional Artifact Auction  | Brass Temple Lamp       |    7000.00 | 2026-10-11 | 2026-10-11 | Upcoming |
|          3 | Royal Furniture Auction       | Royal Wooden Chest      |  150000.00 | 2026-10-12 | 2026-10-12 | Upcoming |
|          4 | Antique Jewellery Auction     | Antique Silver Necklace |   10000.00 | 2026-10-13 | 2026-10-13 | Upcoming |
|          5 | International Antique Auction | Persian Ceramic Vase    |   12000.00 | 2026-10-14 | 2026-10-14 | Upcoming |
|          6 | Historical Mechanical Objects | Vintage Typewriter      |    3500.00 | 2026-10-15 | 2026-10-15 | Upcoming |
+------------+-------------------------------+-------------------------+-----------+------------+------------+----------+
6 rows in set (0.0013 sec)
MYSQL >
```

Figure 5.15: CREATE VIEW auction_details

6. DML STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE WITH INSERTED VALUES

Sample records were inserted into each table using INSERT statements and verified with SELECT queries.

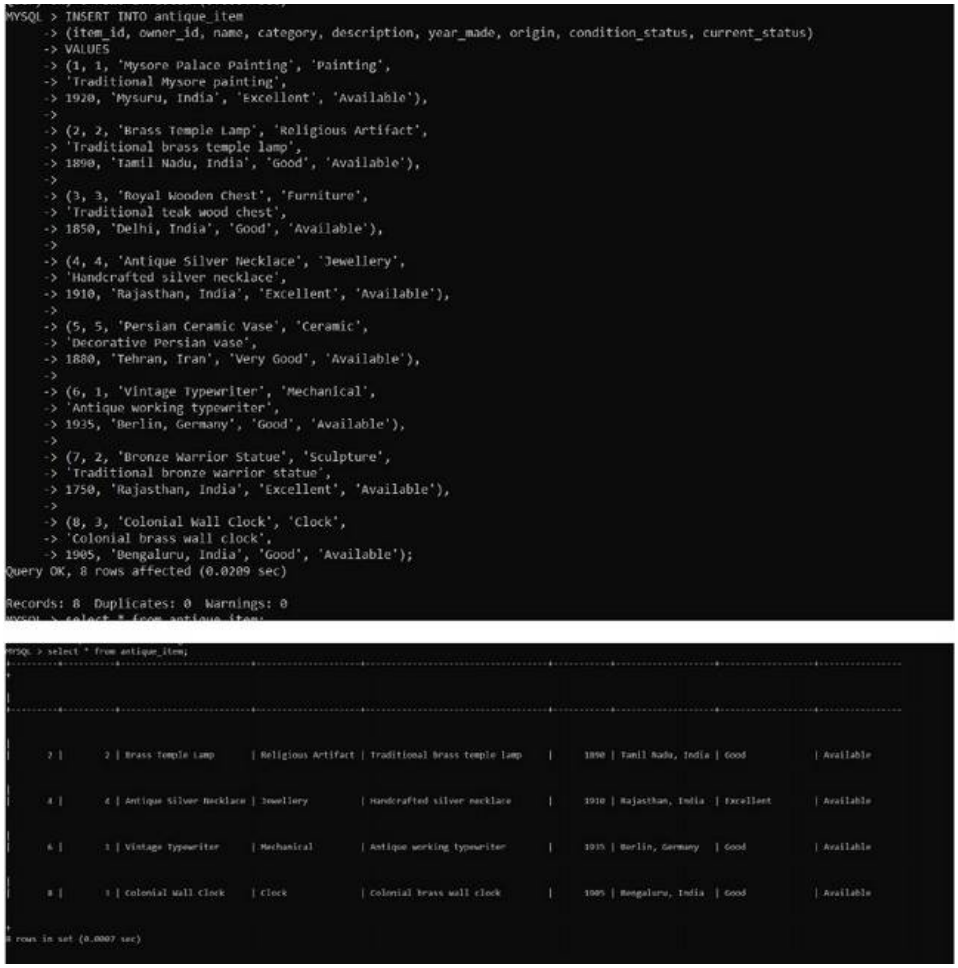
6.1 Insert data into OWNER

```
MYSQL > INSERT INTO owner
     -> (owner_id, name, phone, email, address)
     -> VALUES
     -> (1, 'Ravi Kumar', '9876500001', 'ravi.kumar@gmail.com', 'Mysuru, Karnataka'),
     -> (2, 'Anita Rao', '9876500002', 'anita.rao@gmail.com', 'Bengaluru, Karnataka'),
     -> (3, 'Vikram Shah', '9876500003', 'vikram.shah@gmail.com', 'Mumbai, Maharashtra'),
     -> (4, 'Meera Joshi', '9876500004', 'meera.joshi@gmail.com', 'Jaipur, Rajasthan'),
     -> (5, 'Arjun Desai', '9876500005', 'arjun.desai@gmail.com', 'New Delhi, Delhi');
Query OK, 5 rows affected (0.0216 sec)

Records: 5  Duplicates: 0  Warnings: 0
MYSQL >
MYSQL > SELECT * FROM owner;
+----------+------------+------------+----------------------+----------------------+
| owner_id | name       | phone      | email                | address              |
+----------+------------+------------+----------------------+----------------------+
|        1 | Ravi Kumar | 9876500001 | ravi.kumar@gmail.com | Mysuru, Karnataka    |
|        2 | Anita Rao  | 9876500002 | anita.rao@gmail.com  | Bengaluru, Karnataka |
|        3 | Vikram Shah| 9876500003 | vikram.shah@gmail.com| Mumbai, Maharashtra  |
|        4 | Meera Joshi| 9876500004 | meera.joshi@gmail.com| Jaipur, Rajasthan    |
|        5 | Arjun Desai| 9876500005 | arjun.desai@gmail.com| New Delhi, Delhi     |
+----------+------------+------------+----------------------+----------------------+
5 rows in set (0.0007 sec)
MYSQL >
```

Figure 6.1: INSERT statements and records of OWNER

6.2 Insert data into ANTIQUE_ITEM



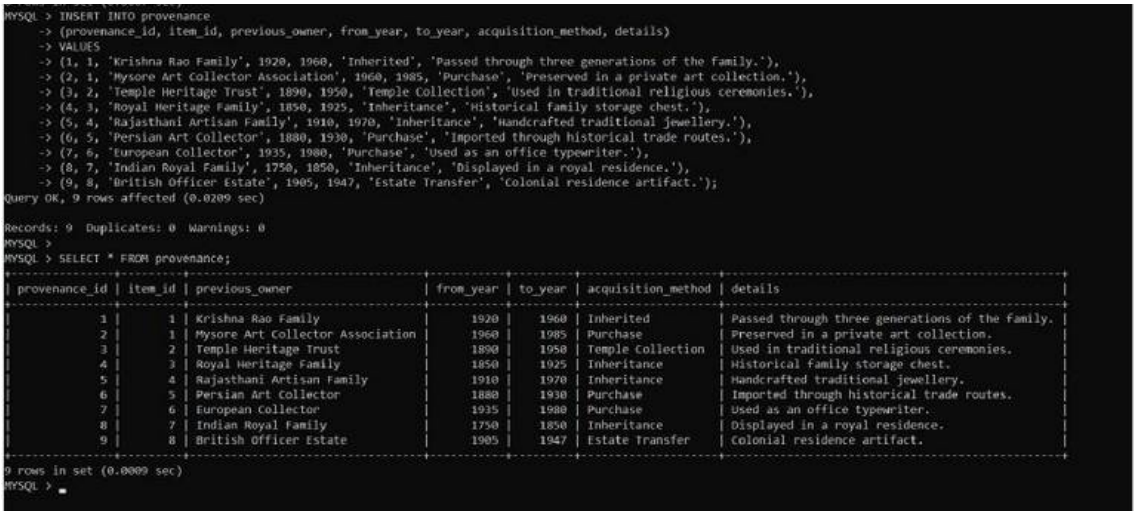
```
MYSQL > INSERT INTO antique_item
     -> (item_id, owner_id, name, category, description, year_made, origin, condition_status, current_status)
     -> VALUES
     -> (1, 1, 'Mysore Palace Painting', 'Painting',
     -> 'Traditional Mysore painting',
     -> 1920, 'Mysuru, India', 'Excellent', 'Available'),
     ->
     -> (2, 2, 'Brass Temple Lamp', 'Religious Artifact',
     -> 'Traditional brass temple lamp',
     -> 1890, 'Tamil Nadu, India', 'Good', 'Available'),
     ->
     -> (3, 3, 'Royal Wooden Chest', 'Furniture',
     -> 'Traditional teak wood chest',
     -> 1850, 'Delhi, India', 'Good', 'Available'),
     ->
     -> (4, 4, 'Antique Silver Necklace', 'Jewellery',
     -> 'Handcrafted silver necklace',
     -> 1910, 'Rajasthan, India', 'Excellent', 'Available'),
     ->
     -> (5, 5, 'Persian Ceramic Vase', 'Ceramic',
     -> 'Decorative Persian vase',
     -> 1880, 'Tehran, Iran', 'Very Good', 'Available'),
     ->
     -> (6, 1, 'Vintage Typewriter', 'Mechanical',
     -> 'Antique working typewriter',
     -> 1935, 'Berlin, Germany', 'Good', 'Available'),
     ->
     -> (7, 2, 'Bronze Warrior Statue', 'Sculpture',
     -> 'Traditional bronze warrior statue',
     -> 1750, 'Rajasthan, India', 'Excellent', 'Available'),
     ->
     -> (8, 3, 'Colonial Wall Clock', 'Clock',
     -> 'Colonial brass wall clock',
     -> 1905, 'Bengaluru, India', 'Good', 'Available');
Query OK, 8 rows affected (0.0209 sec)

Records: 8  Duplicates: 0  Warnings: 0
```

```
MYSQL > select * from antique_item;
+---------+----------+-----------------------+-----------------+-------------------------------+----------+-----------------+-----------------+----------------+
|         |          |                       |                 |                               |          |                 |                 |                |
+---------+----------+-----------------------+-----------------+-------------------------------+----------+-----------------+-----------------+----------------+
|       2 |        2 | Brass Temple Lamp     | Religious Artifact | Traditional brass temple lamp |     1890 | Tamil Nadu, India | Good          | Available      |
|       4 |        4 | Antique Silver Necklace | Jewellery       | Handcrafted silver necklace   |     1910 | Rajasthan, India  | Excellent     | Available      |
|       6 |        1 | Vintage Typewriter    | Mechanical      | Antique working typewriter    |     1935 | Berlin, Germany   | Good          | Available      |
|       8 |        3 | Colonial Wall Clock   | Clock           | Colonial brass wall clock     |     1905 | Bengaluru, India  | Good          | Available      |
+---------+----------+-----------------------+-----------------+-------------------------------+----------+-----------------+-----------------+----------------+
8 rows in set (0.0007 sec)
```

Figure 6.2: INSERT statements and records of ANTIQUE_ITEM

6.3 Insert data into PROVENANCE



```
MYSQL > INSERT INTO provenance
     -> (provenance_id, item_id, previous_owner, from_year, to_year, acquisition_method, details)
     -> VALUES
     -> (1, 1, 'Krishna Rao Family', 1920, 1960, 'Inherited', 'Passed through three generations of the family.'),
     -> (2, 1, 'Mysore Art Collector Association', 1960, 1985, 'Purchase', 'Preserved in a private art collection.'),
     -> (3, 2, 'Temple Heritage Trust', 1890, 1950, 'Temple Collection', 'Used in traditional religious ceremonies.'),
     -> (4, 3, 'Royal Heritage Family', 1850, 1925, 'Inheritance', 'Historical family storage chest.'),
     -> (5, 4, 'Rajasthani Artisan Family', 1910, 1970, 'Inheritance', 'Handcrafted traditional jewellery.'),
     -> (6, 5, 'Persian Art Collector', 1880, 1930, 'Purchase', 'Imported through historical trade routes.'),
     -> (7, 6, 'European Collector', 1935, 1980, 'Purchase', 'Used as an office typewriter.'),
     -> (8, 7, 'Indian Royal Family', 1750, 1850, 'Inheritance', 'Displayed in a royal residence.'),
     -> (9, 8, 'British Officer Estate', 1905, 1947, 'Estate Transfer', 'Colonial residence artifact.');
Query OK, 9 rows affected (0.0209 sec)

Records: 9  Duplicates: 0  Warnings: 0
MYSQL >
MYSQL > SELECT * FROM provenance;
+---------------+---------+-------------------------------+-----------+---------+--------------------+-----------------------------------------+
| provenance_id | item_id | previous_owner                | from_year | to_year | acquisition_method | details                                 |
+---------------+---------+-------------------------------+-----------+---------+--------------------+-----------------------------------------+
|             1 |       1 | Krishna Rao Family            |      1920 |    1960 | Inherited          | Passed through three generations of the family. |
|             2 |       1 | Mysore Art Collector Association |      1960 |    1985 | Purchase           | Preserved in a private art collection.  |
|             3 |       2 | Temple Heritage Trust         |      1890 |    1950 | Temple Collection  | Used in traditional religious ceremonies. |
|             4 |       3 | Royal Heritage Family         |      1850 |    1925 | Inheritance        | Historical family storage chest.        |
|             5 |       4 | Rajasthani Artisan Family     |      1910 |    1970 | Inheritance        | Handcrafted traditional jewellery.      |
|             6 |       5 | Persian Art Collector         |      1880 |    1930 | Purchase           | Imported through historical trade routes. |
|             7 |       6 | European Collector            |      1935 |    1980 | Purchase           | Used as an office typewriter.           |
|             8 |       7 | Indian Royal Family           |      1750 |    1850 | Inheritance        | Displayed in a royal residence.         |
|             9 |       8 | British Officer Estate        |      1905 |    1947 | Estate Transfer    | Colonial residence artifact.            |
+---------------+---------+-------------------------------+-----------+---------+--------------------+-----------------------------------------+
9 rows in set (0.0009 sec)
MYSQL >
```

Figure 6.3: INSERT statements and records of PROVENANCE

6.4 Insert data into VALUATION

```
MYSQL > INSERT INTO valuation
    -> (valuation_id, item_id, estimated_value, valuation_date,
    -> expert_id, expert_name, method, remarks)
    -> VALUES
    -> (1, 1, 250000.00, '2026-01-10', 101, 'Dr. Rajesh Kumar',
    -> 'Historical and Artistic Analysis', 'Authentic Mysore style painting.'),
    ->
    -> (2, 2, 85000.00, '2026-02-15', 102, 'Prof. Anand Rao',
    -> 'Material and Historical Analysis', 'Original brass construction.'),
    ->
    -> (3, 3, 175000.00, '2026-03-05', 103, 'Dr. Vikram Singh',
    -> 'Historical Valuation', 'Teak wood verified.'),
    ->
    -> (4, 4, 120000.00, '2026-03-20', 104, 'Ms. Neha Shah',
    -> 'Material and Craftsmanship', 'Silver authenticity confirmed.'),
    ->
    -> (5, 5, 145000.00, '2026-04-12', 105, 'Dr. Amir Khan',
    -> 'International Market Comparison', 'Persian origin verified.'),
    ->
    -> (6, 6, 4500.00, '2026-04-20', 106, 'Mr. Daniel Thomas',
    -> 'Condition and Rarity', 'Functional antique typewriter.'),
    ->
    -> (7, 7, 300000.00, '2026-05-01', 107, 'Dr. Kavya Rao',
    -> 'Historical and Artistic Analysis', 'Bronze composition verified.'),
    ->
    -> (8, 8, 95000.00, '2026-05-15', 108, 'Mr. Suresh Iyer',
    -> 'Mechanical and Historical Analysis', 'Original mechanism preserved.');
Query OK, 8 rows affected (0.0036 sec)

Records: 8  Duplicates: 0  Warnings: 0
MYSQL >
MYSQL > SELECT * FROM valuation;
+--------------+---------+-----------------+----------------+------------+-------------------+-------------------------------+-------------------------------+
| valuation_id | item_id | estimated_value | valuation_date | expert_id | expert_name       | method                        | remarks                       |
+--------------+---------+-----------------+----------------+------------+-------------------+-------------------------------+-------------------------------+
|            1 |       1 |       250000.00 | 2026-01-10     |       101 | Dr. Rajesh Kumar  | Historical and Artistic Analysis | Authentic Mysore style painting. |
|            2 |       2 |        85000.00 | 2026-02-15     |       102 | Prof. Anand Rao   | Material and Historical Analysis | Original brass construction.  |
|            3 |       3 |       175000.00 | 2026-03-05     |       103 | Dr. Vikram Singh  | Historical Valuation          | Teak wood verified.           |
|            4 |       4 |       120000.00 | 2026-03-20     |       104 | Ms. Neha Shah     | Material and Craftsmanship    | Silver authenticity confirmed. |
|            5 |       5 |       145000.00 | 2026-04-12     |       105 | Dr. Amir Khan     | International Market Comparison | Persian origin verified.    |
|            6 |       6 |         4500.00 | 2026-04-20     |       106 | Mr. Daniel Thomas | Condition and Rarity          | Functional antique typewriter. |
|            7 |       7 |       300000.00 | 2026-05-01     |       107 | Dr. Kavya Rao     | Historical and Artistic Analysis | Bronze composition verified.  |
|            8 |       8 |        95000.00 | 2026-05-15     |       108 | Mr. Suresh Iyer   | Mechanical and Historical Analysis | Original mechanism preserved. |
+--------------+---------+-----------------+----------------+------------+-------------------+-------------------------------+-------------------------------+
```

Figure 6.4: INSERT statements and records of VALUATION

6.5 Insert data into AUCTION

```
MYSQL > INSERT INTO auction
     -> (auction_id, item_id, name, description,
     -> start_date, end_date, location, base_price, status)
     -> VALUES
     -> (1, 1, 'Mysore Heritage Auction',
     -> 'Auction of a traditional Mysore Palace painting.',
     -> '2026-10-10', '2026-10-10', 'Bengaluru Auction Hall', 20000.00, 'Upcoming'),
     ->
     -> (2, 2, 'Traditional Artifact Auction',
     -> 'Auction of a historical brass temple lamp.',
     -> '2026-10-11', '2026-10-11', 'Mysuru Heritage Centre', 7000.00, 'Upcoming'),
     ->
     -> (3, 3, 'Royal Furniture Auction',
     -> 'Auction of a historical royal wooden chest.',
     -> '2026-10-12', '2026-10-12', 'Bengaluru Auction Hall', 150000.00, 'Upcoming'),
     ->
     -> (4, 4, 'Antique Jewellery Auction',
     -> 'Auction of a traditional silver necklace.',
     -> '2026-10-13', '2026-10-13', 'Mumbai Heritage Hall', 10000.00, 'Upcoming'),
     ->
     -> (5, 5, 'International Antique Auction',
     -> 'Auction of a Persian ceramic vase.',
     -> '2026-10-14', '2026-10-14', 'Online', 12000.00, 'Upcoming'),
     ->
     -> (6, 6, 'Historical Mechanical Objects',
     -> 'Auction of a vintage mechanical typewriter.',
     -> '2026-10-15', '2026-10-15', 'Bengaluru Auction Hall', 3500.00, 'Upcoming');
Query OK, 6 rows affected (0.0074 sec)

Records: 6  Duplicates: 0  Warnings: 0
MYSQL >
```

```
MYSQL > SELECT * FROM auction;
+------------+---------+-------------------------+-----------------------------------------+------------+------------+-------------------------+------------+
| auction_id | item_id | name                    | description                             | start_date | end_date   | location                | base_price |
status   | auction_type |
+------------+---------+-------------------------+-----------------------------------------+------------+------------+-------------------------+------------+
---------+--------------+
|          1 |       1 | Mysore Heritage Auction       | Auction of a traditional Mysore Palace painting. | 2026-10-10 | 2026-10-10 | Bengaluru Auction Hall |   20000.00 |
Upcoming | Physical     |
|          2 |       2 | Traditional Artifact Auction  | Auction of a historical brass temple lamp.       | 2026-10-11 | 2026-10-11 | Mysuru Heritage Centre |    7000.00 |
Upcoming | Physical     |
|          3 |       3 | Royal Furniture Auction       | Auction of a historical royal wooden chest.      | 2026-10-12 | 2026-10-12 | Bengaluru Auction Hall |  150000.00 |
Upcoming | Physical     |
|          4 |       4 | Antique Jewellery Auction     | Auction of a traditional silver necklace.        | 2026-10-13 | 2026-10-13 | Mumbai Heritage Hall   |   10000.00 |
Upcoming | Physical     |
|          5 |       5 | International Antique Auction | Auction of a Persian ceramic vase.              | 2026-10-14 | 2026-10-14 | Online                 |   12000.00 |
Upcoming | Physical     |
|          6 |       6 | Historical Mechanical Objects | Auction of a vintage mechanical typewriter.      | 2026-10-15 | 2026-10-15 | Bengaluru Auction Hall |    3500.00 |
Upcoming | Physical     |
+------------+---------+-------------------------+-----------------------------------------+------------+------------+-------------------------+------------+
---------+--------------+
6 rows in set (0.0016 sec)
MYSQL >
```

Figure 6.5: INSERT statements and records of AUCTION

6.6 Insert data into BIDDER

```
MYSQL > INSERT INTO bidder
     -> (bidder_id, name, phone, email, address, registration_date)
     -> VALUES
     -> (1, 'Rahul Sharma', '9000000001', 'rahul@gmail.com',
     -> 'Bengaluru, Karnataka', '2026-08-01'),
     ->
     -> (2, 'Sneha Patil', '9000000002', 'sneha@gmail.com',
     -> 'Mysuru, Karnataka', '2026-08-02'),
     ->
     -> (3, 'Karan Mehta', '9000000003', 'karan@gmail.com',
     -> 'Mumbai, Maharashtra', '2026-08-03'),
     ->
     -> (4, 'Aisha Khan', '9000000004', 'aisha@gmail.com',
     -> 'Hyderabad, Telangana', '2026-08-04'),
     ->
     -> (5, 'Vivek Rao', '9000000005', 'vivek@gmail.com',
     -> 'Chennai, Tamil Nadu', '2026-08-05');
Query OK, 5 rows affected (0.0397 sec)

Records: 5  Duplicates: 0  Warnings: 0
MYSQL >
MYSQL > SELECT * FROM bidder;
+-----------+-------------+------------+-----------------+----------------------+------------------+
| bidder_id | name        | phone      | email           | address              | registration_date |
+-----------+-------------+------------+-----------------+----------------------+------------------+
|         1 | Rahul Sharma | 9000000001 | rahul@gmail.com | Bengaluru, Karnataka | 2026-08-01       |
|         2 | Sneha Patil  | 9000000002 | sneha@gmail.com | Mysuru, Karnataka    | 2026-08-02       |
|         3 | Karan Mehta  | 9000000003 | karan@gmail.com | Mumbai, Maharashtra  | 2026-08-03       |
|         4 | Aisha Khan   | 9000000004 | aisha@gmail.com | Hyderabad, Telangana | 2026-08-04       |
|         5 | Vivek Rao    | 9000000005 | vivek@gmail.com | Chennai, Tamil Nadu  | 2026-08-05       |
+-----------+-------------+------------+-----------------+----------------------+------------------+
5 rows in set (0.0007 sec)
MYSQL >
```

Figure 6.6: INSERT statements and records of BIDDER

6.7 Insert data into BID

```
MYSQL > INSERT INTO bid
    -> (bid_id, auction_id, bidder_id, bid_amount, bid_time)
    -> VALUES
    -> (1, 1, 1, 210000.00, '2026-10-10 10:30:00'),
    -> (2, 1, 2, 220000.00, '2026-10-10 11:00:00'),
    -> (3, 1, 3, 235000.00, '2026-10-10 11:30:00'),
    ->
    -> (4, 2, 1, 75000.00, '2026-10-11 10:30:00'),
    -> (5, 2, 2, 80000.00, '2026-10-11 11:00:00'),
    -> (6, 2, 3, 85000.00, '2026-10-11 11:30:00'),
    ->
    -> (7, 3, 2, 160000.00, '2026-10-12 10:30:00'),
    -> (8, 3, 4, 170000.00, '2026-10-12 11:00:00'),
    -> (9, 3, 5, 180000.00, '2026-10-12 11:30:00'),
    ->
    -> (10, 4, 1, 105000.00, '2026-10-13 11:00:00'),
    -> (11, 4, 3, 115000.00, '2026-10-13 11:30:00'),
    -> (12, 4, 4, 125000.00, '2026-10-13 12:00:00'),
    ->
    -> (13, 5, 2, 125000.00, '2026-10-14 11:00:00'),
    -> (14, 5, 5, 145000.00, '2026-10-14 12:00:00');
Query OK, 14 rows affected (0.0206 sec)

Records: 14  Duplicates: 0  Warnings: 0
MYSQL >
MYSQL > SELECT * FROM bid;
+--------+------------+-----------+------------+---------------------+
| bid_id | auction_id | bidder_id | bid_amount | bid_time            |
+--------+------------+-----------+------------+---------------------+
|      1 |          1 |         1 |  210000.00 | 2026-10-10 10:30:00 |
|      2 |          1 |         2 |  220000.00 | 2026-10-10 11:00:00 |
|      3 |          1 |         3 |  235000.00 | 2026-10-10 11:30:00 |
|      4 |          2 |         1 |   75000.00 | 2026-10-11 10:30:00 |
|      5 |          2 |         2 |   80000.00 | 2026-10-11 11:00:00 |
|      6 |          2 |         3 |   85000.00 | 2026-10-11 11:30:00 |
|      7 |          3 |         2 |  160000.00 | 2026-10-12 10:30:00 |
|      8 |          3 |         4 |  170000.00 | 2026-10-12 11:00:00 |
|      9 |          3 |         5 |  180000.00 | 2026-10-12 11:30:00 |
|     10 |          4 |         1 |  105000.00 | 2026-10-13 11:00:00 |
|     11 |          4 |         3 |  115000.00 | 2026-10-13 11:30:00 |
|     12 |          4 |         4 |  125000.00 | 2026-10-13 12:00:00 |
|     13 |          5 |         2 |  125000.00 | 2026-10-14 11:00:00 |
```

Figure 6.7: INSERT statements and records of BID

6.8 Insert data into TRANSACTION

```
MYSQL > INSERT INTO `transaction`
    -> (transaction_id, auction_id, winning_bidder_id, final_price, transaction_date)
    -> VALUES
    -> (1, 1, 3, 235000.00, '2026-10-10'),
    -> (2, 2, 3, 85000.00, '2026-10-11'),
    -> (3, 3, 5, 180000.00, '2026-10-12');
Query OK, 3 rows affected (0.0358 sec)

Records: 3  Duplicates: 0  Warnings: 0
MYSQL >
MYSQL > SELECT * FROM `transaction`;
+----------------+------------+-------------------+-------------+------------------+
| transaction_id | auction_id | winning_bidder_id | final_price | transaction_date |
+----------------+------------+-------------------+-------------+------------------+
|              1 |          1 |                 3 |   235000.00 | 2026-10-10       |
|              2 |          2 |                 3 |    85000.00 | 2026-10-11       |
|              3 |          3 |                 5 |   180000.00 | 2026-10-12       |
+----------------+------------+-------------------+-------------+------------------+
3 rows in set (0.0006 sec)
MYSQL >
```

Figure 6.8: INSERT statements and records of TRANSACTION

6.9 Insert data into PAYMENT

```
MYSQL > INSERT INTO payment
    -> (payment_id, transaction_id, amount, payment_date, payment_method, status)
    -> VALUES
    -> (1, 1, 235000.00, '2026-10-10', 'UPI', 'Completed'),
    -> (2, 2, 85000.00, '2026-10-11', 'Card', 'Completed'),
    -> (3, 3, 180000.00, '2026-10-12', 'Bank Transfer', 'Completed');
Query OK, 3 rows affected (0.0408 sec)

Records: 3  Duplicates: 0  Warnings: 0
MYSQL >
MYSQL > SELECT * FROM payment;
+------------+----------------+-----------+--------------+----------------+-----------+
| payment_id | transaction_id | amount    | payment_date | payment_method | status    |
+------------+----------------+-----------+--------------+----------------+-----------+
|          1 |              1 | 235000.00 | 2026-10-10   | UPI            | Completed |
|          2 |              2 |  85000.00 | 2026-10-11   | Card           | Completed |
|          3 |              3 | 180000.00 | 2026-10-12   | Bank Transfer  | Completed |
+------------+----------------+-----------+--------------+----------------+-----------+
3 rows in set (0.0006 sec)
MYSQL >
```

Figure 6.9: INSERT statements and records of PAYMENT

7. QUERIES

7.1 SIMPLE QUERY WITH GROUP BY, AGRREGATE

Joins, GROUP BY, HAVING and aggregate functions.

JOIN

Joins AUCTION, ANTIQUE_ITEM, BID and BIDDER to list every bid with the auction name, item name, bidder name, amount and time, sorted by highest bid first.

```
MYSQL > SELECT
    ->     a.name AS auction_name,
    ->     ai.name AS item_name,
    ->     b.name AS bidder_name,
    ->     bid.bid_amount,
    ->     bid.bid_time
    -> FROM auction a
    -> JOIN antique_item ai ON a.item_id = ai.item_id
    -> JOIN bid ON a.auction_id = bid.auction_id
    -> JOIN bidder b ON bid.bidder_id = b.bidder_id
    -> ORDER BY bid.bid_amount DESC;
+-----------------------------+-----------------------+-------------+------------+---------------------+
| auction_name                | item_name             | bidder_name | bid_amount | bid_time            |
+-----------------------------+-----------------------+-------------+------------+---------------------+
| Mysore Heritage Auction     | Mysore Palace Painting | Karan Mehta |  235000.00 | 2026-10-10 11:30:00 |
| Mysore Heritage Auction     | Mysore Palace Painting | Sneha Patil |  220000.00 | 2026-10-10 11:00:00 |
| Mysore Heritage Auction     | Mysore Palace Painting | Rahul Sharma |  210000.00 | 2026-10-10 10:30:00 |
| Royal Furniture Auction     | Royal Wooden Chest    | Vivek Rao   |  180000.00 | 2026-10-12 11:30:00 |
| Royal Furniture Auction     | Royal Wooden Chest    | Aisha Khan  |  170000.00 | 2026-10-12 11:00:00 |
| Royal Furniture Auction     | Royal Wooden Chest    | Sneha Patil |  160000.00 | 2026-10-12 10:30:00 |
| International Antique Auction | Persian Ceramic Vase | Vivek Rao   |  145000.00 | 2026-10-14 12:00:00 |
| International Antique Auction | Persian Ceramic Vase | Sneha Patil |  125000.00 | 2026-10-14 11:00:00 |
| Antique Jewellery Auction   | Antique Silver Necklace | Aisha Khan  |  125000.00 | 2026-10-13 12:00:00 |
| Antique Jewellery Auction   | Antique Silver Necklace | Karan Mehta |  115000.00 | 2026-10-13 11:30:00 |
| Antique Jewellery Auction   | Antique Silver Necklace | Rahul Sharma |  105000.00 | 2026-10-13 11:00:00 |
| Traditional Artifact Auction | Brass Temple Lamp     | Karan Mehta |   85000.00 | 2026-10-11 11:30:00 |
| Traditional Artifact Auction | Brass Temple Lamp     | Sneha Patil |   80000.00 | 2026-10-11 11:00:00 |
| Traditional Artifact Auction | Brass Temple Lamp     | Rahul Sharma |   75000.00 | 2026-10-11 10:30:00 |
+-----------------------------+-----------------------+-------------+------------+---------------------+
14 rows in set (0.0014 sec)
MYSQL >
```

Figure 7.1: JOIN query – all bids with auction, item and bidder

GROUP BY and COUNT

Counts the total number of bids received in each auction.

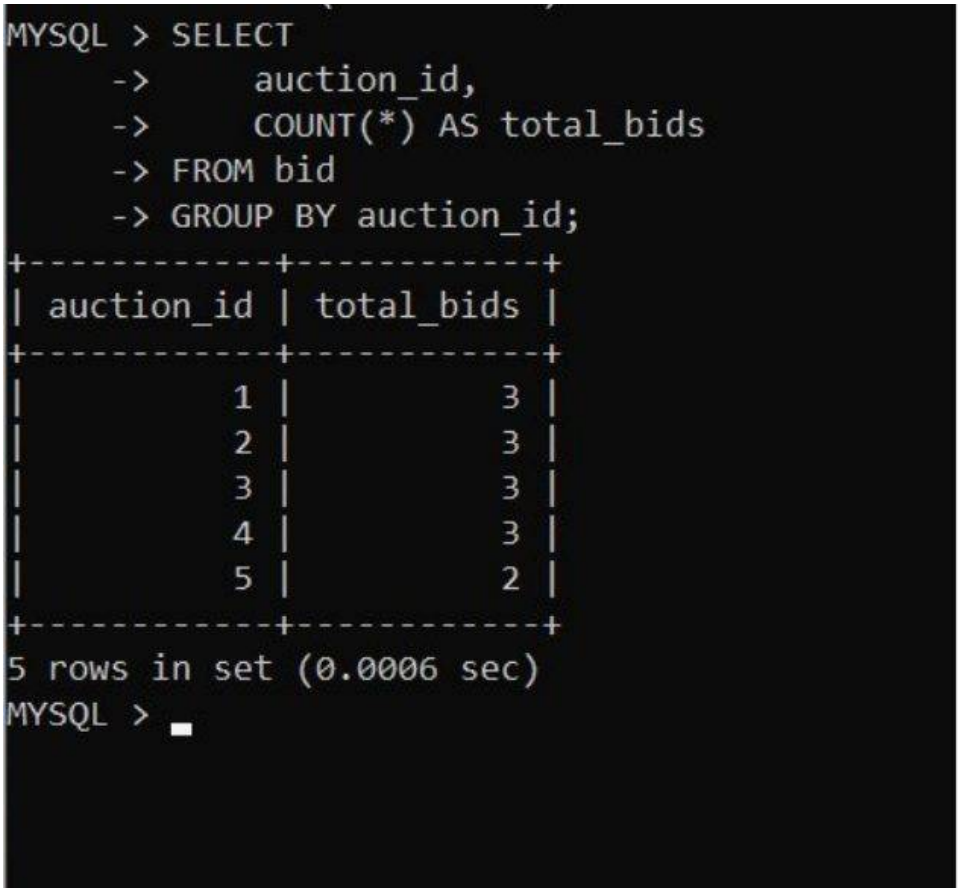


Figure 7.2: GROUP BY with COUNT – bids per auction

GROUP BY, SUM and HAVING

Shows auctions whose total bid amount is greater than 300000.

```
MYSQL > SELECT
     ->     auction_id,
     ->     SUM(bid_amount) AS total_bid_amount
     -> FROM bid
     -> GROUP BY auction_id
     -> HAVING SUM(bid_amount) > 300000;
+------------+------------------+
| auction_id | total_bid_amount |
+------------+------------------+
|          1 |        665000.00 |
|          3 |        510000.00 |
|          4 |        345000.00 |
+------------+------------------+
3 rows in set (0.0007 sec)
MYSQL >
```

Figure 7.3: GROUP BY, SUM and HAVING

Aggregate functions

Uses COUNT, SUM, AVG, MIN and MAX on the bid amounts.

```
MYSQL > SELECT
     ->     COUNT(bid_amount) AS total_bids,
     ->     SUM(bid_amount) AS total_bid_amount,
     ->     AVG(bid_amount) AS average_bid,
     ->     MIN(bid_amount) AS lowest_bid,
     ->     MAX(bid_amount) AS highest_bid
     -> FROM bid;
+------------+------------------+--------------+------------+-------------+
| total_bids | total_bid_amount | average_bid  | lowest_bid | highest_bid |
+------------+------------------+--------------+------------+-------------+
|         14 |       2030000.00 | 145000.000000 |   75000.00 |   235000.00 |
+------------+------------------+--------------+------------+-------------+
1 row in set (0.0007 sec)
MYSQL >
```

Figure 7.4: Aggregate functions on bid amounts

Highest bid for each auction

Uses MAX with GROUP BY to find the highest bid in every auction.

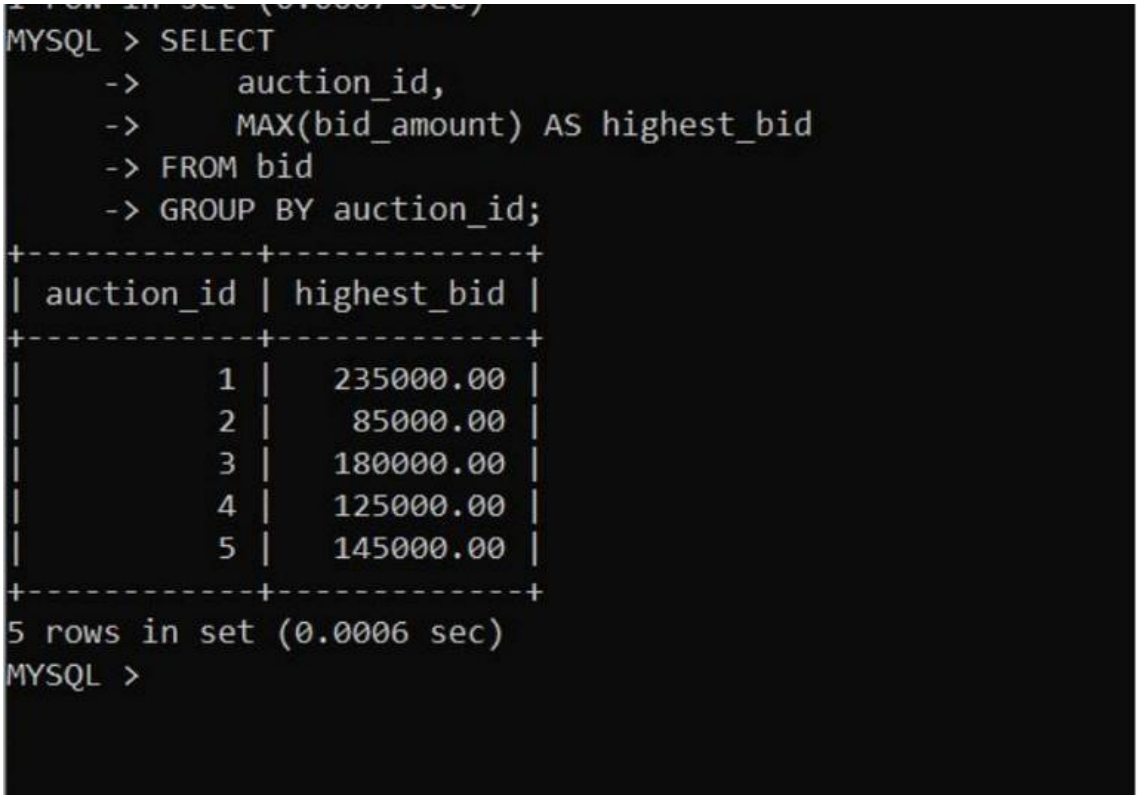


Figure 7.5: MAX with GROUP BY – highest bid per auction

7.2 UPDATE OPERATION

A payment record is updated inside a transaction (START TRANSACTION) and the change is saved with COMMIT.

```
MYSQL > START TRANSACTION;
Query OK, 0 rows affected (0.0007 sec)
MYSQL >
MYSQL > UPDATE payment
     -> SET status = 'Processing'
     -> WHERE payment_id = 1;
Query OK, 1 row affected (0.0016 sec)

Rows matched: 1  Changed: 1  Warnings: 0
MYSQL >
MYSQL > SELECT *
     -> FROM payment
     -> WHERE payment_id = 1;
+------------+----------------+-----------+--------------+----------------+------------+
| payment_id | transaction_id | amount    | payment_date | payment_method | status     |
+------------+----------------+-----------+--------------+----------------+------------+
|          1 |              1 | 235000.00 | 2026-10-10   | UPI            | Processing |
+------------+----------------+-----------+--------------+----------------+------------+
1 row in set (0.0006 sec)
MYSQL > COMMIT;
Query OK, 0 rows affected (0.0038 sec)
MYSQL > SELECT *
     -> FROM payment
     -> WHERE payment_id = 1;
+------------+----------------+-----------+--------------+----------------+------------+
| payment_id | transaction_id | amount    | payment_date | payment_method | status     |
+------------+----------------+-----------+--------------+----------------+------------+
|          1 |              1 | 235000.00 | 2026-10-10   | UPI            | Processing |
+------------+----------------+-----------+--------------+----------------+------------+
1 row in set (0.0006 sec)
MYSQL >
```

Figure 7.6: UPDATE inside START TRANSACTION and COMMIT

7.3 DELETE OPERATION

A test bidder is inserted and then deleted with DELETE; the final SELECT returns an empty set, confirming the row is removed.

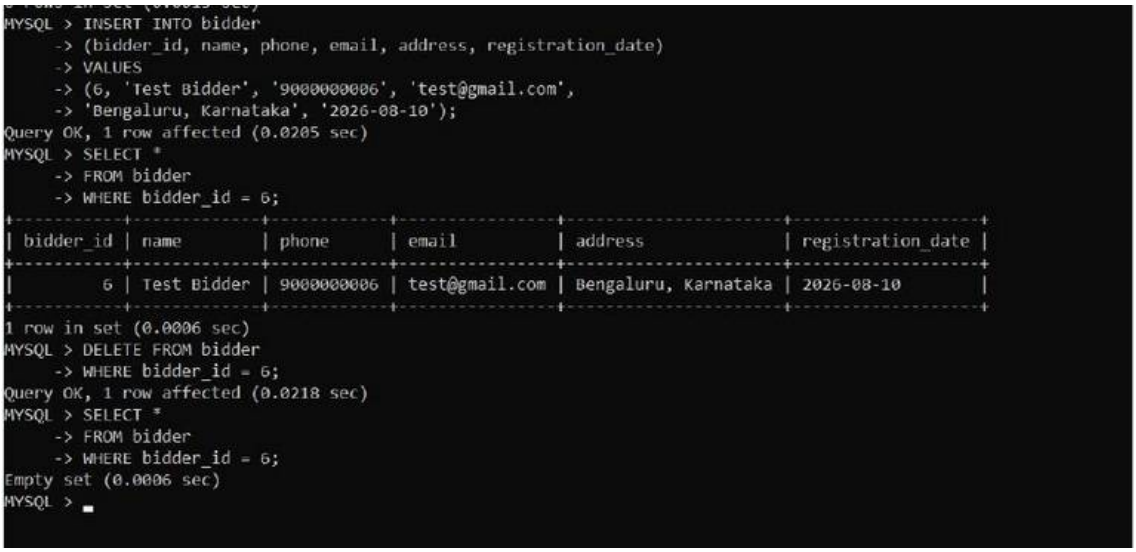


Figure 7.7: DELETE operation

7.4 CORRELATED QUERY

A correlated sub-query refers to a column of the outer query, so it is evaluated once for every outer row. The query below lists, for each auction, the bid(s) equal to that auction's highest bid.

```
SELECT b.bid_id, b.auction_id, b.bidder_id, b.bid_amount
FROM bid b
WHERE b.bid_amount = (SELECT MAX(b2.bid_amount)
                      FROM bid b2
                      WHERE b2.auction_id = b.auction_id);
```

[Output screenshot of this query to be added.]

7.5 NESTED QUERY

A nested (sub-) query lists the items whose estimated value is above the average estimated value.

```
MYSQL > SELECT
     ->     item_id,
     ->     estimated_value
     -> FROM valuation
     -> WHERE estimated_value >
     -> (
     ->     SELECT AVG(estimated_value)
     ->     FROM valuation
     -> );
+---------+-----------------+
| item_id | estimated_value |
+---------+-----------------+
|       1 |       250000.00 |
|       3 |       175000.00 |
|       7 |       300000.00 |
+---------+-----------------+
3 rows in set (0.0009 sec)
MYSQL >
```

Figure 7.8: Nested query – items above the average estimated value

8. STORED PROCEDURES, FUCNTIONS AND TRIGGERS

8.1 STORED PROCEDURES OR FUNCTIONS

Stored procedure

The procedure GetAuctionDetails() returns the main details of all auctions. SHOW PROCEDURE STATUS confirms that it exists in the database.

```
MYSQL > DELIMITER //
MYSQL >
MYSQL > CREATE PROCEDURE GetAuctionDetails()
    -> BEGIN
    ->     SELECT
    ->         auction_id,
    ->         name,
    ->         base_price,
    ->         start_date,
    ->         end_date,
    ->         status
    ->     FROM auction;
    -> END //
ERROR: 1304: PROCEDURE GetAuctionDetails already exists
MYSQL >
MYSQL > DELIMITER ;
MYSQL > CALL GetAuctionDetails();
+------------+--------------------------------+------------+------------+------------+----------+
| auction_id | name                           | base_price | start_date | end_date   | status   |
+------------+--------------------------------+------------+------------+------------+----------+
|          1 | Mysore Heritage Auction        |   20000.00 | 2026-10-10 | 2026-10-10 | Upcoming |
|          2 | Traditional Artifact Auction   |    7000.00 | 2026-10-11 | 2026-10-11 | Upcoming |
|          3 | Royal Furniture Auction        |  150000.00 | 2026-10-12 | 2026-10-12 | Upcoming |
|          4 | Antique Jewellery Auction      |   10000.00 | 2026-10-13 | 2026-10-13 | Upcoming |
|          5 | International Antique Auction |   12000.00 | 2026-10-14 | 2026-10-14 | Upcoming |
|          6 | Historical Mechanical Objects  |    3500.00 | 2026-10-15 | 2026-10-15 | Upcoming |
+------------+--------------------------------+------------+------------+------------+----------+
6 rows in set (0.0009 sec)

Query OK, 0 rows affected (0.0009 sec)
MYSQL >
```

Figure 8.1: Creating and calling the procedure GetAuctionDetails()

```
MYSQL > SHOW PROCEDURE STATUS
    -> WHERE Db = DATABASE();
+-----------------+-------------------+-----------+----------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| Db              | Name              | Type      | Definer  | Modified            | Created             | Security_type | Comment | character_set_client | collation_connection | Database Collation |
+-----------------+-------------------+-----------+----------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| antique_auction_db | GetAuctionDetails | PROCEDURE | root@localhost | 2026-10-02 23:28:31 | 2026-10-02 23:28:31 | DEFINER       |         | utf8mb4              | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci |
+-----------------+-------------------+-----------+----------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
1 row in set (0.0019 sec)
MYSQL >
```

Figure 8.2: SHOW PROCEDURE STATUS

Function

CalculateFutureValue() returns the given value increased by 10%. It is applied to the estimated value of every item in VALUATION.

```
MYSQL > DELIMITER //
MYSQL >
MYSQL > CREATE FUNCTION CalculateFutureValue(current_value DECIMAL(12,2))
     -> RETURNS DECIMAL(12,2)
     -> DETERMINISTIC
     -> BEGIN
     ->     RETURN current_value * 1.10;
     -> END //
ERROR: 1304: FUNCTION CalculateFutureValue already exists
MYSQL >
MYSQL > DELIMITER ;
MYSQL > SELECT CalculateFutureValue(100000) AS future_value;
+--------------+
| future_value |
+--------------+
|    110000.00 |
+--------------+
1 row in set (0.0007 sec)
MYSQL >
```

Figure 8.3: Creating the function CalculateFutureValue()

```
MYSQL > SELECT
     ->     item_id,
     ->     estimated_value,
     ->     CalculateFutureValue(estimated_value) AS future_value
     -> FROM valuation;
+---------+-----------------+--------------+
| item_id | estimated_value | future_value |
+---------+-----------------+--------------+
|       1 |       250000.00 |    275000.00 |
|       2 |        85000.00 |     93500.00 |
|       3 |       175000.00 |    192500.00 |
|       4 |       120000.00 |    132000.00 |
|       5 |       145000.00 |    159500.00 |
|       6 |         4500.00 |      4950.00 |
|       7 |       300000.00 |    330000.00 |
|       8 |        95000.00 |    104500.00 |
+---------+-----------------+--------------+
8 rows in set (0.0007 sec)
MYSQL >
```

Figure 8.4: Using the function on the VALUATION table

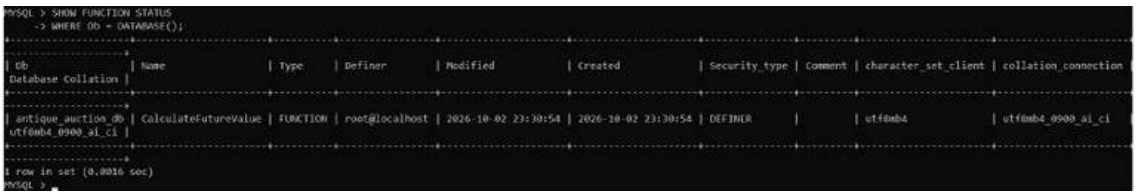


Figure 8.5: SHOW FUNCTION STATUS

8.2 TRIGGERS

The trigger **after_auction_insert** runs after a new auction is inserted and sets the item's current_status to 'Auctioned'.

```
MYSQL > DELIMITER //
MYSQL >
MYSQL > CREATE TRIGGER after_auction_insert
     -> AFTER INSERT ON auction
     -> FOR EACH ROW
     -> BEGIN
     ->     UPDATE antique_item
     ->     SET current_status = 'Auctioned'
     ->     WHERE item_id = NEW.item_id;
     -> END //
ERROR: 1359: Trigger already exists
MYSQL >
MYSQL > DELIMITER ;
MYSQL > _
```

Figure 8.6: Creating the trigger after_auction_insert

```
MYSQL > DELIMITER ;
MYSQL > INSERT INTO auction
    -> (
    ->     auction_id,
    ->     item_id,
    ->     name,
    ->     description,
    ->     start_date,
    ->     end_date,
    ->     location,
    ->     base_price,
    ->     status,
    ->     auction_type
    -> )
    -> VALUES
    -> (
    ->     7,
    ->     7,
    ->     'Test Trigger Auction',
    ->     'Temporary auction to test trigger',
    ->     '2026-11-01',
    ->     '2026-11-01',
    ->     'Bengaluru',
    ->     250000.00,
    ->     'Upcoming',
    ->     'Physical'
    -> );
Query OK, 1 row affected (0.0240 sec)
MYSQL >
```

Figure 8.7: Testing the trigger – a test auction is inserted for item 7

```
MYSQL > SELECT
    ->     item_id,
    ->     name,
    ->     current_status
    -> FROM antique_item
    -> WHERE item_id = 7;
+---------+----------------------+----------------+
| item_id | name                 | current_status |
+---------+----------------------+----------------+
|       7 | Bronze Warrior Statue | Auctioned      |
+---------+----------------------+----------------+
1 row in set (0.0007 sec)
MYSQL >
```

Figure 8.8: Trigger result – current_status of item 7 is now 'Auctioned'

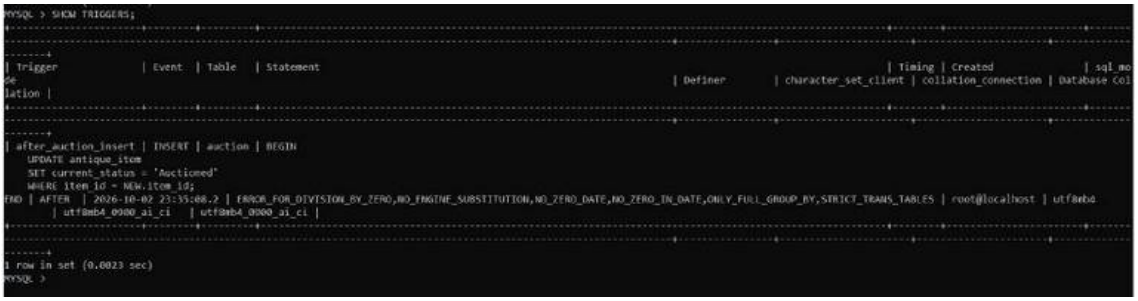


Figure 8.9: SHOW TRIGGERS – trigger on the AUCTION table

9. FRONT END DEVELOPEMNT

The database was implemented in MySQL and operated through the MySQL command-line client for all DDL, DML, query, procedure, function and trigger work shown in the earlier chapters. On top of it, a web front end named **Antique Auction** was developed (running locally on localhost:3000). It lets a visitor browse antiques and auctions, register and sign in, place bids, view their bids and transactions, and give feedback, while an admin dashboard shows summary reports of the stored data. The screens of the application are shown below.

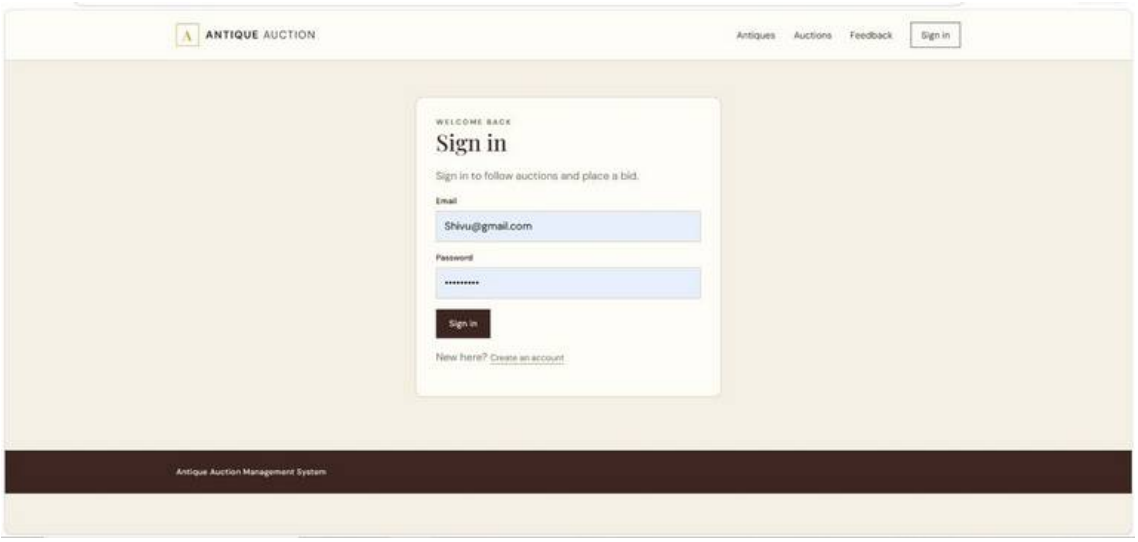


Figure 9.1: Sign in page – an existing user signs in with email and password.

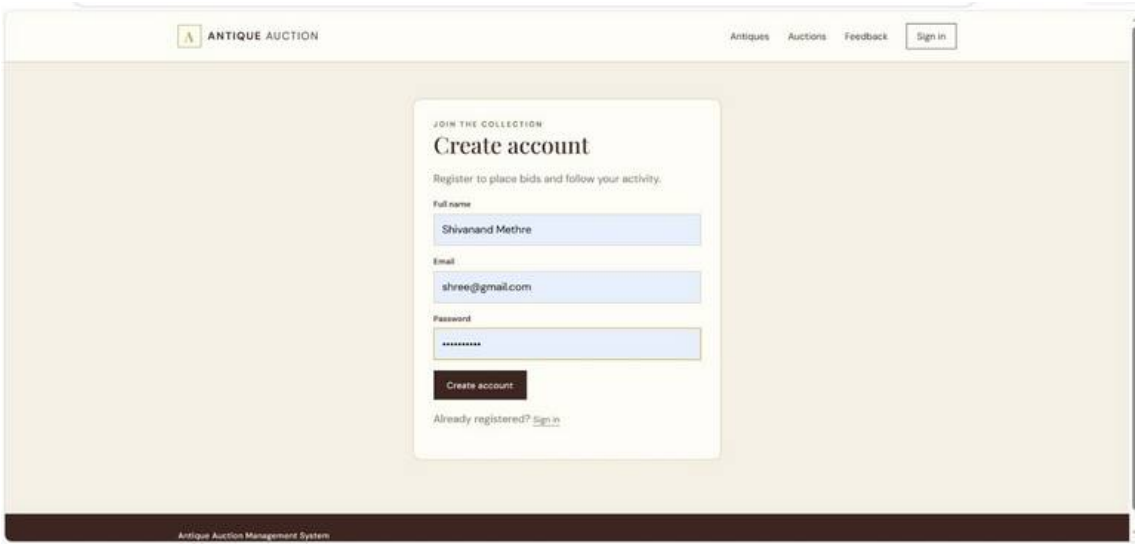


Figure 9.2: Create account page – a new user registers with full name, email and password.

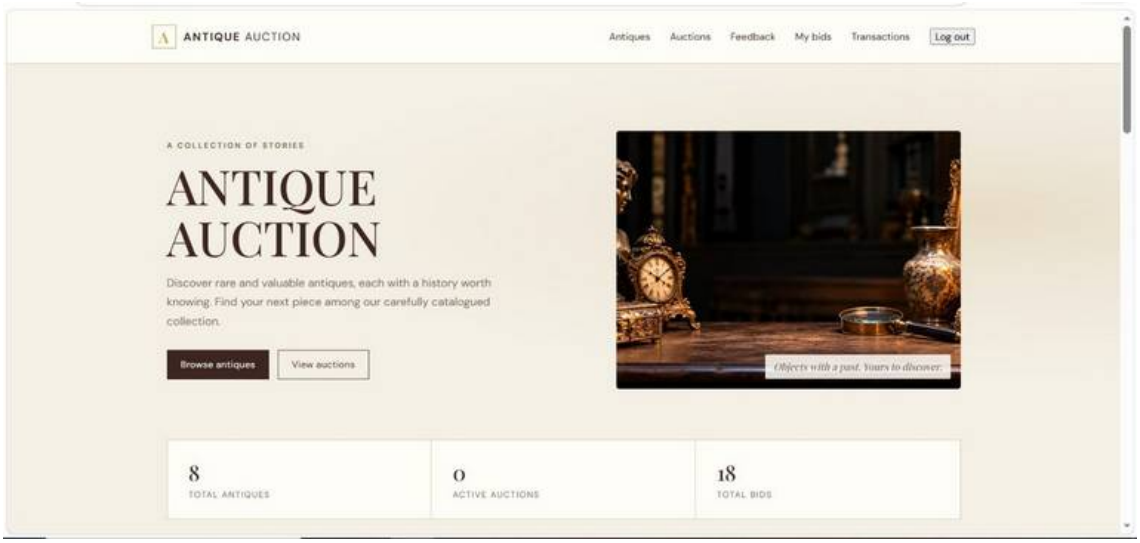


Figure 9.3: Home page – introduction with the total antiques, active auctions and total bids.

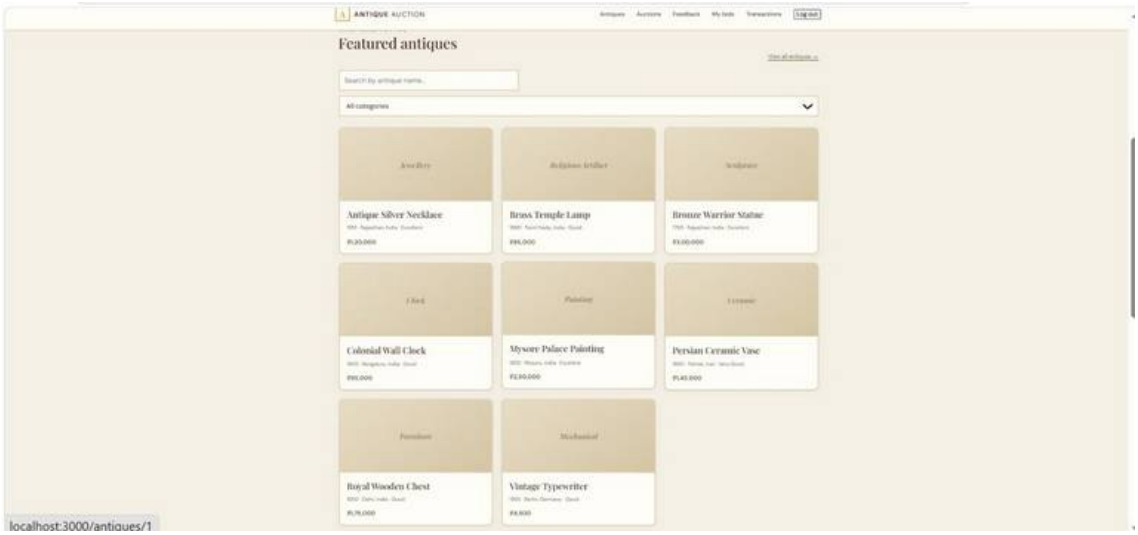


Figure 9.4: Featured antiques on the home page, with search by name and a category filter.

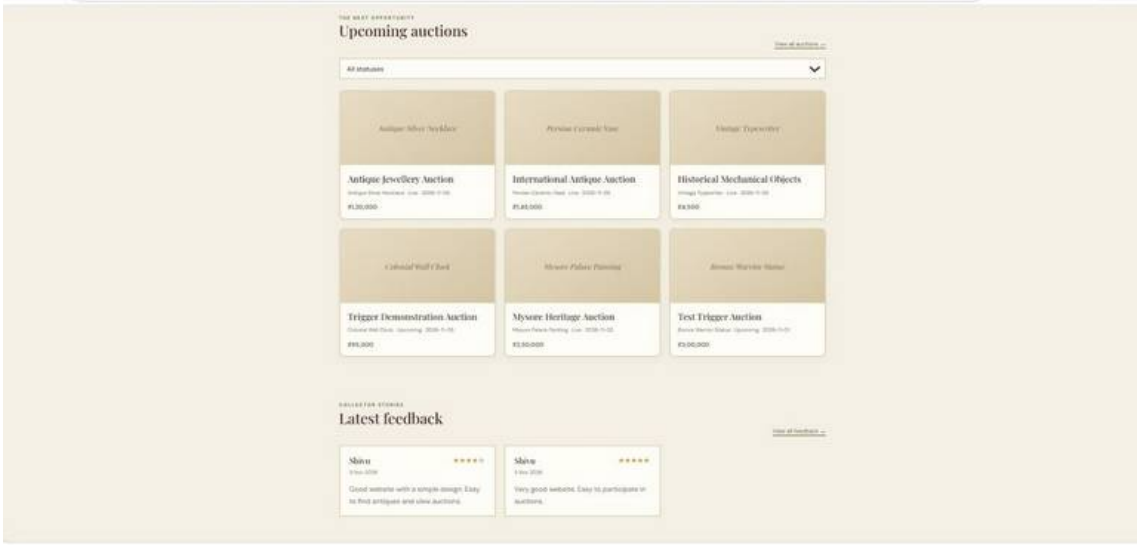


Figure 9.5: Upcoming auctions (with status filter) and latest feedback on the home page.

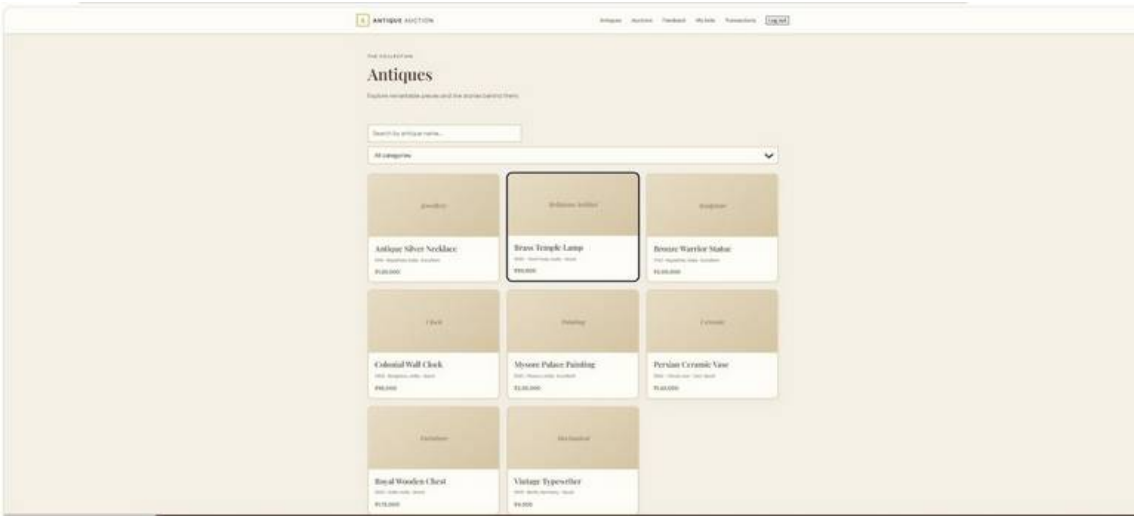


Figure 9.6: Antiques page – all antiques with year, origin, condition and estimated value.

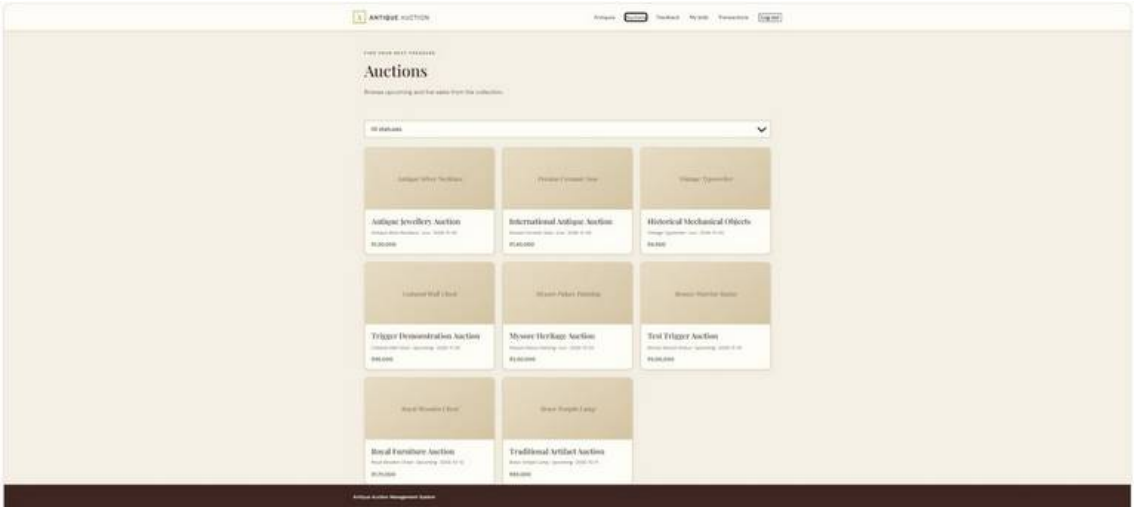


Figure 9.7: Auctions page – upcoming and live auctions with a status filter.

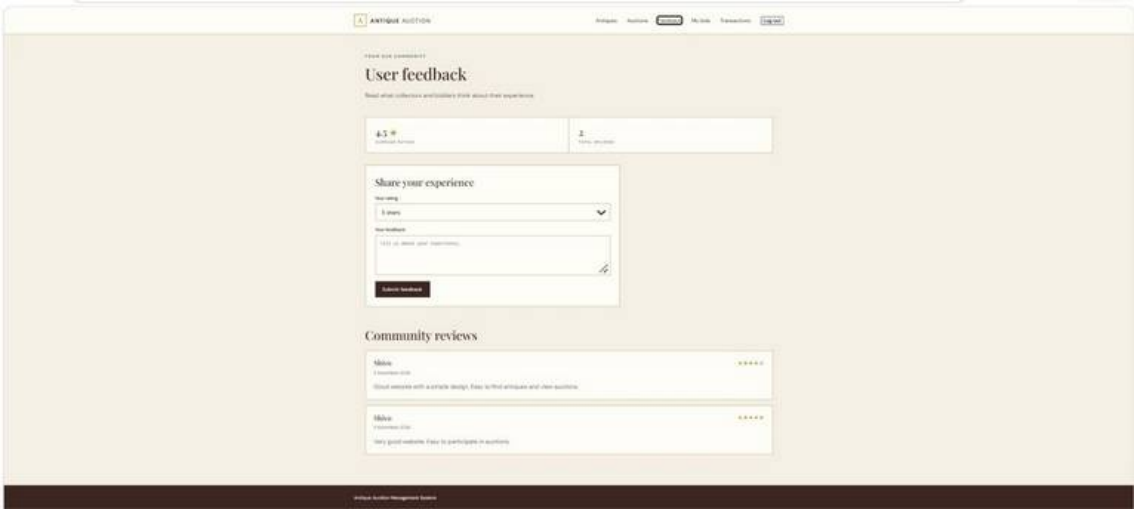


Figure 9.8: User feedback page – average rating, a form to submit feedback, and community reviews.



Figure 9.9: Auction details page – item details, previous highest bid, minimum next bid and the Place a bid form.

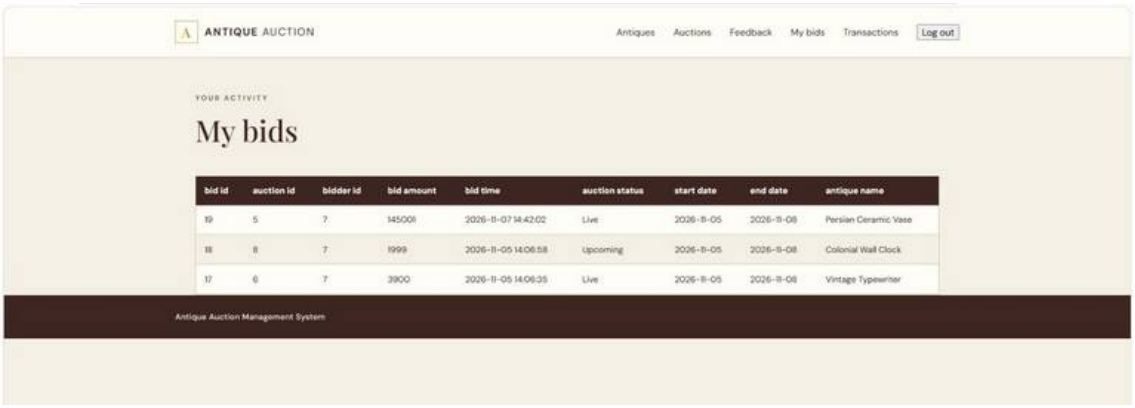


Figure 9.10: My bids page – the bids placed by the signed-in user.

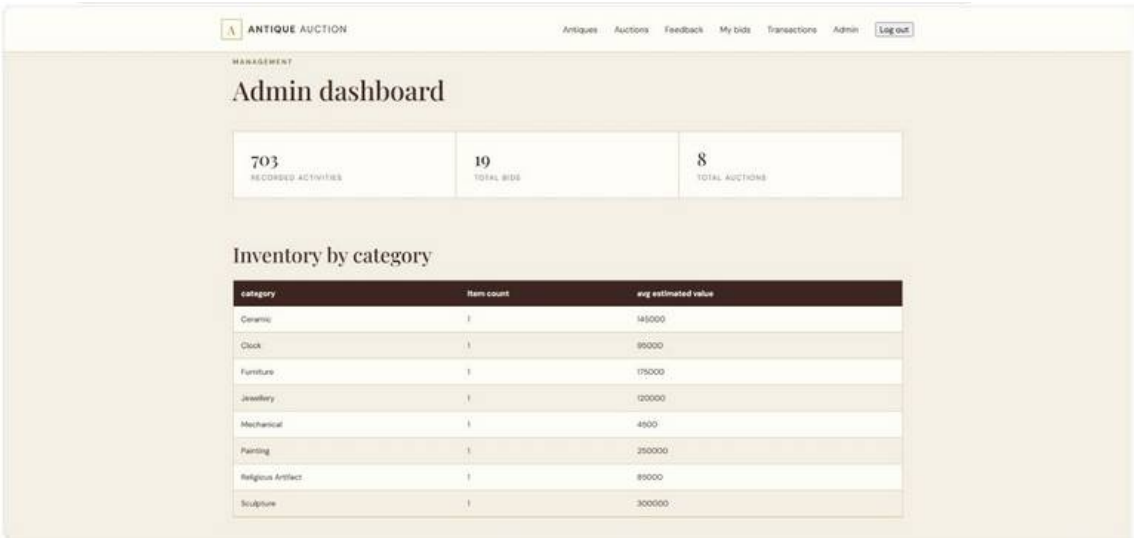


Figure 9.11: Admin dashboard – recorded activities, total bids and total auctions, and inventory by category.

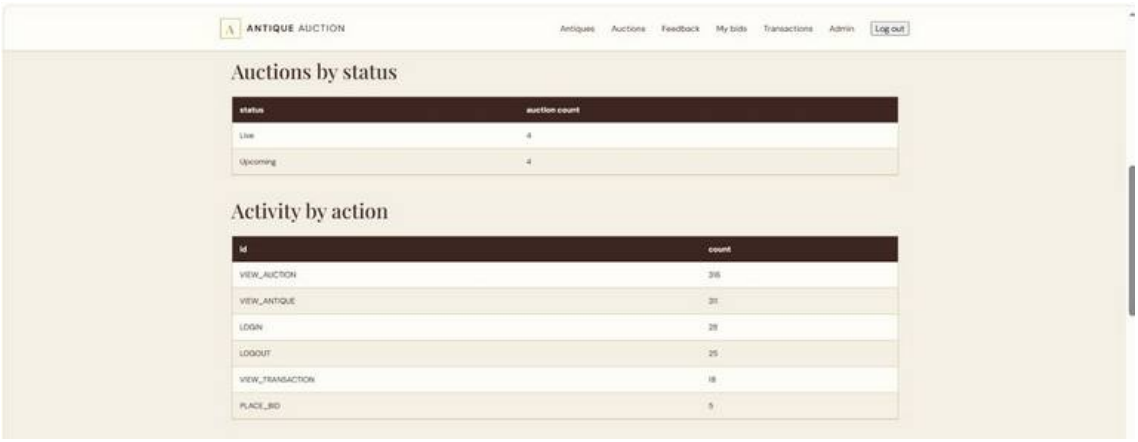


Figure 9.12: Admin dashboard – auctions by status and activity by action.

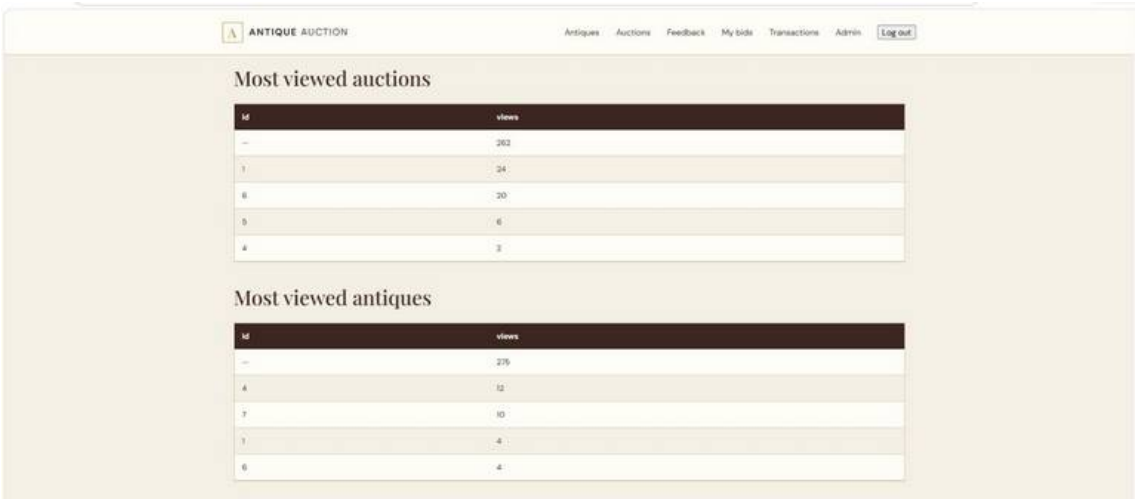


Figure 9.13: Admin dashboard – most viewed auctions and most viewed antiques.

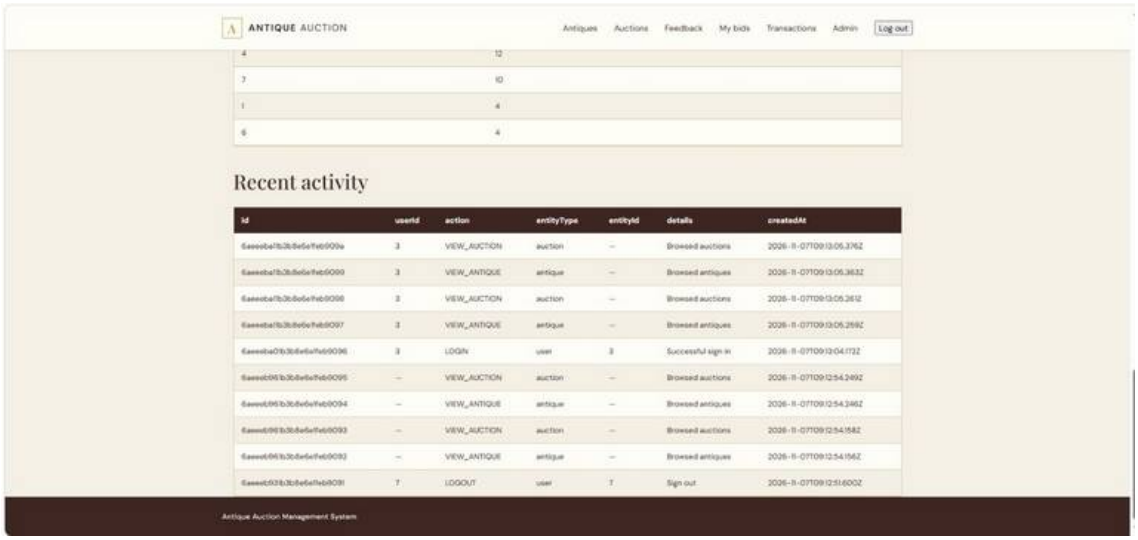


Figure 9.14: Admin dashboard – recent activity log (login, logout and page views).

GitHub Link: https://github.com/shreenandan30/Digital-Antique-Auction-Platform

APPENDIX A
DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Table A.1: Definitions, acronyms and abbreviations

Term	Meaning
DBMS	Database Management System
RDBMS	Relational Database Management System
SQL	Structured Query Language
DDL	Data Definition Language (CREATE, ALTER, DROP)
DML	Data Manipulation Language (INSERT, UPDATE, DELETE, SELECT)
ER	Entity-Relationship
PK	Primary Key
FK	Foreign Key
Provenance	The documented history of ownership and origin of an antique
Valuation	An expert estimate of the monetary value of an antique
Bid	An offer of money made by a bidder for an item in an auction
Trigger	A stored program that runs automatically on an INSERT, UPDATE or DELETE event
Stored Procedure	A named, reusable block of SQL stored in the database
View	A virtual table defined by a stored SELECT query
SRN	Student Registration Number